Python Handbook — Interview Edition

How to use this edition

The book to re-read the night before an interview: chapters 1–20 of the *Python Handbook* (Python 3.14, compatible with 3.12+) cut down to the facts, numbers, questions and traps that interviews test. Nothing here is new — every line comes from the full book; open the matching chapter there for the explanation, the runnable script and its real output.

- **Same layout in every chapter:** **Must-know** (one-line definitions) → **Key facts & numbers** (tables) → **Code to remember** (snippets you may be asked to write) → **In the interview** (likely questions, short answers) → **Gotchas** (mistake → correct idea) → **TL;DR**.
- **Night before:** read the *Top 25 questions* below, then every chapter's **In the interview** and **Gotchas**, then the *Last-minute checklist* at the end.
- **A week before:** read each **Must-know** and **Key facts** table; say each **In the interview** answer out loud in about 30 seconds; for anything shaky, go back to the full chapter.
- **Coding rounds:** Chapter 19's pattern table maps problem wording to a pattern and the Python structure to use; its gotchas list the bugs that pass samples and fail hidden tests. Chapter 20 is the consolidated **Gotchas** list for the whole language.
- **Versions** are marked where they matter: *(3.10)* means added in Python 3.10. Check which version the judge or interviewer's environment runs before using new helpers.
- Chapter 21 of the full book (cheat sheets and glossary) is not repeated here; its tables are folded into the chapters.

Top 25 questions across the book

#	Question	One-line answer	Ch
1	Compiled or interpreted?	Both: source → AST → bytecode, run by a stack VM; 3.11+ specialises bytecode by type	1, 17
2	Dynamic vs strong typing?	Types live on objects (names can re-point); no silent cross-type coercion (`"3" + 4` raises)	2
3	`is` vs `==` ?	Identity vs value (`__eq__`); `is` only for `None` /sentinels; small-int caching is an implementation detail	2, 17
4	Mutable vs immutable; why must keys be hashable?	list/dict/set change in place; keys are stored by hash, which must never change	2, 6
5	Why is `0.1 + 0.2 != 0.3` ?	Binary IEEE-754 floats; use `math.isclose` , `Decimal("0.1")` for money	2
6	How is a list implemented?	Dynamic array of pointers; O(1) index/append (amortised), O(n) front insert and `in`	5
7	How does a dict work; is it ordered?	Open-addressing hash table ≤ 2/3 full, O(1) avg; insertion-ordered since 3.7	6
8	Shallow vs deep copy?	`copy()` / `[:]` copies the outer container only; `deepcopy` recurses (handles cycles)	5
9	Mutable default argument bug?	Defaults are evaluated once at `def` ; use `None` and create inside	7, 20
10	LEGB and `UnboundLocalError` ?	Local → Enclosing → Global → Built-in; any assignment makes a name local to the whole function	7
11	What is a closure; late binding?	Inner function keeping outer variables; captures variables, not values → `lambda i=i: i`	7
12	How does a decorator work?	`@d` = `f = d(f)` at definition; wrapper `(*args, **kwargs)` + `functools.wraps` ; factories for arguments	7, 12
13	`*args` , `**kwargs` , `/` , `*` ?	Extra positionals (tuple) / keywords (dict); positional-only before `/` , keyword-only after `*`	7
14	Iterator vs iterable vs generator?	Iterable gives a fresh iterator; iterator is one-shot `__next__` ; generator = lazy iterator from `yield`	11
15	How does `with` work?	`__enter__` → body → `__exit__` always (truthy return suppresses); `@contextmanager` with `try/finally`	9, 12
16	`__repr__` vs `__str__` ; `__eq__` and `__hash__` ?	repr for developers, str falls back to it; equal ⇒ same hash, `__eq__` alone makes a class unhashable	10
17	What is the MRO; what does `super()` call?	C3 linearization; `super()` = next class in the MRO, enabling cooperative multiple inheritance	10
18	ABC vs Protocol vs duck typing?	Nominal and enforced at instantiation / structural, checked statically / behaviour at call time	10, 13
19	What does `@dataclass` generate?	`__init__` , `__repr__` , `__eq__` ; `frozen` , `order` , `slots` ; `field(default_factory=list)`	10
20	Are type hints enforced?	No — static checkers (mypy, pyright) and libraries (pydantic) read them	13
21	What is the GIL?	One thread runs bytecode at a time in default CPython; released on I/O; free-threaded build optional in 3.14	16

#	Question	One-line answer	Ch
22	Threads vs processes vs asyncio?	Blocking I/O / CPU-bound pure Python / thousands of async connections	16
23	How does CPython free memory; can it leak?	Refcounting + generational cycle GC; leaks = objects still referenced	17
24	How do you speed up slow Python?	Profile → better algorithm/structure → cache → vectorise → native code or parallelism	17
25	EAFP vs LBYL?	Python prefers try/except: race-free and cheap when failure is rare	9, 20

1. Getting Started with Python

What Python is, how CPython runs code, environments, basic syntax, comments/docstrings, PEP 8 and PEP 20.

Must-know

- **Python** — high-level, **dynamically but strongly typed**, garbage-collected, multi-paradigm language. Guido van Rossum, 1989–91; Python 3 (2008) is the only live line; Python 2 EOL 1 Jan 2020.
- **CPython** — the reference implementation (written in C). Others: **PyPy** (JIT, often several times faster on long-running pure Python), **MicroPython** (microcontrollers), GraalPy/Jython (JVM).
- **Compiled or interpreted?** — both: source → tokens → **AST** → **bytecode** (code object) → stack-based **VM**. The whole file compiles first, so a syntax error on line 200 stops line 1 from running.
- `.pyc` / `__pycache__` — cached bytecode for *imported* modules; header (magic number + timestamp/size or hash) + serialised code object. Not source protection; don't commit it.
- **Specialising interpreter (3.11+)** — CPython rewrites instructions based on observed types; the big 3.11 speed-up.
- **GIL** — lock in the default CPython build letting only one thread run bytecode at a time.
- **PEP** — Python Enhancement Proposal: design document for a change or standard. **PEP 8** = style, **PEP 20** = Zen of Python.
- **Governance** — five-person **Steering Council** since 2019; one feature release every October (PEP 602).
- **venv** — per-project environment with its own `site-packages`; `sys.prefix` points to it; stdlib is shared with the base interpreter.
- `python -m pip` — guarantees pip belongs to the same interpreter as `python`.
- **Soft keyword** — reserved only in context (`match` , `case` , `_` , `type`), so old code using `match` as a name still works.
- **Docstring** — first statement string of a module/class/function, kept in `__doc__`, read by `help()` . `#` comments are discarded by the tokenizer. No block-comment syntax.
- **Pythonic** — using the language's idioms (direct iteration, context managers, comprehensions) instead of translating another language.

Key facts & numbers

Command	Does
`python3 -i script.py`	run file, then REPL with its variables
`python3 -m module`	run a module on the import path (`-m venv` , `-m pip` , `-m http.server` , `-m pytest`)
`python3 -c "..."`	one-line program
`dis.dis(func)`	show bytecode

PEP 8 rule	Example
4 spaces, no tabs; lines ≤ 79 (72 for comments/docstrings)	teams often relax to 88–100
`snake_case` funcs/vars/modules, `CapWords` classes, `UPPER_SNAKE` constants, `_internal`	`load_data`, `UserProfile`, `MAX_RETRIES`
Imports at top, grouped: stdlib, third-party, local	—
`is None`, not `== None`; `if ready:` not `== True`	—
Continue lines inside brackets, not `\`	a trailing space after `\` breaks it

In the interview

Q: Describe Python in 30 seconds. **A:** High-level, dynamically but strongly typed, garbage-collected, multi-paradigm; CPython compiles to bytecode and runs it on a stack VM; trades raw speed for productivity and pushes heavy work into C extensions like NumPy.

Q: Why is Python slow and how do teams cope? **A:** Every operation is dynamically type-checked in the VM. Vectorise with NumPy, write C/Rust extensions, use PyPy, or rely on CPython's 3.11+ speed-ups.

Q: How do you set up a project? **A:** `python -m venv .venv`, activate, `python -m pip install`, pin in `requirements.txt` / `pyproject.toml`; `.venv/` in `.gitignore`.

Q: Is a triple-quoted string a comment? **A:** No — an evaluated-and-discarded string expression, unless first in a body, where it becomes the docstring.

Q: When not to use Python? **A:** Hard real-time, browser front-ends, native mobile, CPU-bound multi-threading on the default GIL build.

Gotchas

- "Interpreted = line by line" → the whole file is compiled to bytecode first.
- Dynamic ≠ weak → Python is dynamic **and** strong (`"3" + 4` raises).
- `sudo pip install` globally → breaks OS tools; use a venv.
- Naming your file `random.py` / `email.py` → shadows the stdlib module on import.
- A bare `2 + 3` in a script prints nothing → only the REPL echoes.
- `input() + 1` → `TypeError`; `input()` always returns `str`.

TL;DR

Source → AST → bytecode → VM; dynamic-but-strong, GC'd, batteries included. One venv per project, `python -m pip`, PEP 8 via a formatter.

2. Variables, Data Types and Operators

Names vs objects, identity vs equality, mutability, typing, numbers, truthiness, conversions, operators and precedence.

Must-know

- **Variable** — a **name bound to an object**, not a box. `=` binds, **never copies**. `del x` removes one reference; the object dies when its refcount hits zero.
- **Identity / type / value** — every object has all three; identity (`id()`, the address in CPython) and type never change; value changes only if the type is **mutable**.
- **`==` vs `is`** — `==` compares values via `__eq__`; `is` compares identity. Use `is` only for singletons: `None`, `True`, `False`, sentinels.
- **Mutable** — list, dict, set, bytearray (change in place; aliases see it). **Immutable** — int, float, str, tuple, frozenset, bytes (a "change" rebinds to a new object).
- **Hashable** — has a stable hash; required for dict keys and set members. A tuple is hashable only if all its items are.

- **Dynamic typing** — types live on objects; a name can point to any type. **Strong typing** — no silent cross-type coercion (except numeric widening bool → int → float → complex).
- **int** — arbitrary precision, never overflows. **float** — IEEE-754 double (≈15–17 digits). **Decimal** (from strings) for money; **Fraction** for exact ratios.
- **Truthiness** — falsy: `None`, `False`, numeric zeros, empty containers; everything else truthy. Custom: `__bool__`, fallback `__len__`.
- `bool` **subclasses** `int` — `True == 1`, so `{1: "a", True: "b"}` is `{1: 'b'}`.
- `and` / `or` — short-circuit and **return an operand**, not necessarily a bool.
- **Walrus** `:=` — assignment expression (binds inside an expression).
- **str vs repr** — `str` for users, `repr` unambiguous for developers.

Key facts & numbers

Expression	Result	Why
`0.1 + 0.2 == 0.3`	`False`	binary floats; use `math.isclose`
`round(2.5)`	`2`	round half to even
`-7 // 2` / `-7 % 2`	`-4` / `1`	`//` floors toward −∞; `%` takes divisor's sign
`int(2.999)`	`2`	truncates toward zero
`int("0x1A", 0)`	`26`	base 0 reads the prefix
`-3 ** 2`, `2 ** 3 ** 2`	`-9`, `512`	`**` binds tighter than unary minus; right-associative
`~2`	`-3`	`~x == -x - 1`
`bool("False")`	`True`	non-empty string
`Decimal(0.1)`	long ugly value	use `Decimal("0.1")`

Precedence (high → low): `**` > unary `+ - ~` > `* @ / // %` > `+ -` > shifts > `&` > `^` > `|` > comparisons (one level, they chain) > `not` > `and` > `or` > `if-else` > `lambda` > `:=`.

Type	Exact?	Use for
int	yes	counts, IDs, money in cents
float	no	science, ML, measurements
Decimal	yes (base 10)	money, tax
Fraction	yes	exact rational maths

Code to remember

```
a = b = []          # one list, two names
a.append(1); print(b)          # [1]
grid = [[0] * 3 for _ in range(3)]   # NOT [[0]*3]*3 (3 refs to one row)
x, y = y, x         # builds tuple (y, x), then unpacks
if value is not None:  # when 0 / "" are valid data
    ...
```

In the interview

Q: What does assignment do? **A:** Binds a name to an object; it never copies. Copy with `a.copy()` / `list(a)` / `a[:]` (shallow) or `copy.deepcopy`.

Q: Why is `256 is 256` True but `1000 is 1000` may not be? **A:** CPython caches small ints −5..256 — an implementation detail; 3.8+ warns on `is` with a literal. Use `==`.

Q: Can a tuple change? **A:** Its slots can't be reassigned, but mutable objects inside can be mutated; such a tuple is unhashable.

Q: Why must dict keys be immutable? **A:** Keys are stored by hash; a changing value would change the hash and lose the key.

Q: Why is `0.1 + 0.2 != 0.3`? **A:** Neither has an exact binary form; the rounded sum differs from rounded 0.3 in the last bit.

Q: `a += [1]` vs `a = a + [1]`? **A:** `+=` mutates the list in place (aliases see it); `+` builds a new list.

Q: Why is `range(10**9)` cheap? **A:** Lazy — computes values from start/stop/step on demand.

Q: `in` on list vs set? **A:** O(n) vs average O(1).

Q: Implicit conversions? **A:** Only up the numeric tower; everything else explicit. Bad content → `ValueError`, wrong type → `TypeError`.

Gotchas

- `if x is 5` / `is "bob"` → use `==`.
- `if x == 1 or 2:` always true → `if x in (1, 2):`.
- `x == 1 & y == 2` parses as a chain around `1 & y` → parenthesise, or use `and`.
- `{}` is an empty **dict** → `set()`.
- `type(x) == int` rejects subclasses → `isinstance(x, int)`.
- `if value:` treats `0` as missing → `is not None`.
- `random` for tokens → `secrets`.
- Shadowing built-ins (`list`, `sum`, `id`, `type`) → pick other names.

TL;DR

Names label objects; `is` for `None` only; know which types are mutable/hashable; floats are approximate (`isclose`, `Decimal("…")`); `//` floors; `and` / `or` return operands; parenthesise when unsure.

3. Control Flow: Decisions and Loops

`if` / `elif`, conditional expressions, `match` / `case`, `for` / `while`, loop control, loop `else`, loop idioms.

Must-know

- **`if` / `elif` / `else`** — conditions tested top-down by truthiness; **first truthy branch wins**, later ones are not evaluated. Order specific → general; prefer flat **guard clauses**.
- **No `switch`** — use `match` (3.10+) or a **dispatch table** (`{"a": f, "b": g}[key]()`).
- **Conditional expression** — `a if cond else b`; keep it short, never nested.
- **`match` / `case`** — subject evaluated once, cases tried in order, **no fall-through**, not a jump table.
- **Capture trap** — a bare name (`case RED:`) **captures**, it doesn't compare. Compare constants with a dotted name (`Color.RED`), a literal or a guard.
- **Iteration protocol** — `for` calls `iter()` then `next()` until `StopIteration`, so it works on any iterable.
- **`while`** — for unknown counts; the body must make progress; `while True` + `break` for loop-and-a-half.
- **`break` / `continue` / `pass`** — leave the innermost loop / next iteration / no-op placeholder. To exit nested loops, move them into a function and `return`.
- **Loop `else`** — runs only when the loop was **not ended by `break`** ("no break"); also runs for an empty iterable; skipped by `return`.
- **Loop variable** survives the loop with its last value (unless the iterable was empty).

Key facts & numbers

Pattern	Example	Note
Literal	`200`, `"Mon"`, `None`	`None` / `True` / `False` compared with `is`
Capture / wildcard	`x` / `_`	always match
Value	`Color.RED`	dotted name → `==`
OR / AS	`p1 | p2`, `pat as name`	alternatives must bind the same names
Sequence	`[first, *rest]`	not `str` / `bytes`
Mapping	`{"type": "click", "pos": p}`	**extra keys ignored**; `**rest` collects
Class	`Point(x=0, y=0)`, `Point(x, 0)`	`isinstance` + attrs; positional needs `__match_args__` (auto for dataclass/namedtuple)
Guard	`case p if cond:`	pattern **and** condition

Instead of	Write
`for i in range(len(xs))`	`for x in xs` / `for i, x in enumerate(xs)`
`a[i], b[i]`	`zip(a, b, strict=True)` (plain `zip` stops at the shortest)
`for k in d: v = d[k]`	`for k, v in d.items()`
flag + loop	`any(...)` / `all(...)` (short-circuit)
read-chunk loop	`while (chunk := f.read(n)):`

Code to remember

```
for user in users:
    if user.name == target:
        print("found"); break
else:                       # no break
    print("not found")

match event:
    case {"type": "click", "pos": (x, y)}: handle(x, y)
    case _: raise ValueError(event)
```

In the interview

Q: Does Python have switch? **A:** `match` (3.10+) does that and destructuring; dicts work as dispatch tables.

Q: What does a loop `else` do? **A:** Runs if the loop finished without `break` — ideal for search/not-found without a flag.

Q: How do you match a constant in `match`? **A:** Dotted name, literal, or guard — a bare name captures.

Q: Do mapping patterns need exact keys? **A:** No, extras are ignored; `**rest` captures them.

Q: Why does removing during iteration skip items? **A:** The list iterator advances by index; removal shifts later items down.

Q: `zip` with unequal lengths? **A:** Stops at shortest; `strict=True` raises `ValueError`.

Gotchas

- `else if` → `elif`; `if flag == True` → `if flag`.
- Expecting fall-through / `break` in a `case` → neither exists.
- `case [a, b]` doesn't match a string.

- No `case _:` → unexpected input silently ignored; consider raising.
- `continue` before the counter update in `while` → infinite loop.
- `while x != 1.0` with floats may never end → use `<` /tolerance.
- Mutating a list/dict while iterating → build a new one or iterate a copy.

TL;DR

First truthy branch wins; `match` = ordered shape-aware patterns, bare names capture; iterate items directly with `enumerate` / `zip(strict=True)` / `.items()` ; loop `else` = "no break".

4. Strings

Immutability, slicing, escapes, f-strings and format specs, methods, efficient joining, Unicode vs bytes, regex.

Must-know

- `str` — immutable, ordered sequence of **Unicode code points**; zero-based and negative indexing. Every "change" returns a new string — rebind the name (`name = name.strip()`).
- **Hashable** because immutable → usable as dict keys; the hash is cached after the first computation.
- **Slicing** `s[start:stop:step]` — stop excluded, bounds clipped (never raises), `s[::-1]` reverses, O(k) copy. Stop is exclusive so `s[:k] + s[k:] == s` and `len(s[a:b]) == b - a` .
- **Raw string** `r"..."` — disables escapes; use for regexes and Windows paths. Can't end in a single `\` . Unknown escapes like `"\d"` give `SyntaxWarning` (3.12+).
- **f-string** (3.6) — default formatter, fastest. `f"{x=}"` (3.8) prints `x=` + `repr(x)` . Custom specs via `__format__` .
- **Format spec** — `[[fill]align][sign][#][0][width][,][.prec][type]` , e.g. `{x:>10,.2f}` .
- **Other styles** — `str.format` when the template is data; `%` for `logging` 's lazy formatting; `string.Template` for **untrusted** templates (`str.format` can walk attributes).
- `"".join(parts)` — O(n) builder; `+=` in a loop is O(n²) worst case (fast only by a CPython in-place accident).
- `str` **vs** `bytes` — text vs raw bytes; Python 3 never guesses an encoding. **Decode early, encode late**, always say `encoding="utf-8"` .
- **UTF-8** — variable 1–4 bytes, ASCII-compatible; `len("café".encode()) == 5` .
- **Mojibake** — bytes decoded with the wrong encoding (`cafÃ©`). Equal-looking strings may differ (composed vs decomposed) → normalise with **NFC**.
- `casefold()` — aggressive lowercase for caseless comparison (`"ß"` → `"ss"`).

Key facts & numbers

Operation	Cost
`len(s)` , `s[i]`	O(1) (CPython stores fixed-width chars)
`s[a:b]`	O(b − a)
`s + t`	O(len s + len t) — always new
`in` , `find` , `count` , `replace` , `split` , `strip`	O(n)
`sep.join(parts)`	O(total) — sizes first, copies once
`s == t`	O(n) worst; O(1) if lengths differ

Method pair	Difference
`find` / `index`	`-1` vs `ValueError` when absent
`split()` / `split(" ")`	no-arg splits on runs of whitespace and drops empties
`isdecimal` ⊂ `isdigit` ⊂ `isnumeric`	`²` is digit, `½` is numeric
`re.search` / `match` / `fullmatch`	anywhere / at start / whole string
`findall` / `finditer`	strings (tuples if several groups) / `Match` objects

Regex	Meaning
`*? +?`	lazy quantifiers
`(?P<name>…)` / `(?:…)`	named group / non-capturing group
`\b`	word boundary (needs `r""` — `"\b"` is backspace)

Code to remember

```
out = []
for row in rows:
    out.append(f"{row.name:<10}{row.price:>8,.2f}")
text = "\n".join(out)
clean = s.translate(str.maketrans("", "", string.punctuation))
m = re.fullmatch(r"(?P<y>\d{4})-(?P<m>\d{2})", s)
if m: year = m["y"]
```

In the interview

Q: If strings are immutable, how does `s += "x"` work? **A:** Builds `s + "x"` and rebinds `s`; CPython may resize in place with one reference — don't rely on it.

Q: Why is `+=` in a loop slow? **A:** Each step copies everything: O(n²). Collect in a list, `"".join` once.

Q: Why not f-strings in log calls? **A:** The string is built even if the level is disabled, and aggregators can't group by template. Use `logger.info("%s", x)`.

Q: Palindrome via `s == s[::-1]`? **A:** O(n) time and O(n) extra space; two pointers use O(1) space.

Q: Why can't regex parse HTML? **A:** It can't handle nesting — use a real parser.

Q: What is catastrophic backtracking? **A:** Nested quantifiers like `(a+)+$` take exponential time on near-matches — the basis of ReDoS.

Q: Why `re.compile`? **A:** Readability and reuse; `re` caches recent patterns, so speed gain is small.

Gotchas

- `if s.find("x"):` → `0` is falsy and `-1` truthy; use `"x" in s`.
- `f"{price:.2}"` = 2 **significant** digits → `:.2f` for decimals.
- `",".join([1, 2])` → `TypeError`; `",".join(map(str, nums))`.
- `"%s" % some_tuple` → wrap as `(some_tuple,)`.
- Sorting names with `<` puts `"Zoe"` before `"adam"` → `key=str.casefold`.
- `len()` counts code points, not user-visible glyphs (emoji with modifiers).
- `errors="ignore"` hides data loss; omitting `encoding=` breaks across OSes.
- Formatting values into SQL → use driver parameters (`?`).
- `m.group()` without checking `m is not None` → `AttributeError`.

TL;DR

Immutable Unicode; slices never raise; f-strings + format spec; list + `join` for building; decode early/encode late in UTF-8; `search` / `match` / `fullmatch`, raw strings, beware backtracking.

5. Lists, Tuples and Arrays

Lists as dynamic arrays, big-O, methods, slicing, comprehensions, sorting, copying, tuples, unpacking, namedtuple, `array`.

Must-know

- **list** — ordered, mutable, heterogeneous; a **dynamic array of pointers** with ~12.5% proportional over-allocation → **amortised O(1) append**.
- **Amortised** — occasional resizes copy everything, but total copying over n appends is O(n).
- `+` **vs** `extend` / `+=` — `+` makes a new list; `extend` / `+=` mutate in place (and `extend` returns `None`).
- `append` **vs** `extend` — `append([3, 4])` adds one nested element; `extend([3, 4])` adds two; `extend("hi")` adds `'h'`, `'i'`.
- **Slice read = shallow copy; slice assignment edits in place** (may change length unless stepped; stepped needs equal length).
- **Comprehension** — `[expr for x in it if cond]`; `if/else` goes before `for`; multiple `for`s read left to right; variables don't leak (Py3). Use a **generator expression** for one-pass consumers (`sum`, `any`, `join`).
- `sort()` **vs** `sorted()` — in place returning `None` vs new list from any iterable. **Timsort**: stable, O(n log n), O(n) on nearly sorted input. `key` is called **once per element**.
- **Stability** — equal elements keep prior order → multi-key sorts via tuple key or successive passes.
- **Copies** — `=` aliases; `copy()` / `list()` / `[:]` shallow (nested objects shared); `copy.deepcopy` recursive (handles cycles via a memo).
- **tuple** — commas make tuples (`(42,)`); shallowly immutable; hashable if contents are; lighter than lists; fixed-shape records.
- **Unpacking** — counts must match (`ValueError`); `first, *rest = seq` → `rest` is always a **list**; only one starred target.
- **namedtuple** / `typing.NamedTuple` — tuple subclass with named fields, `_asdict`, `_replace`; equal to plain tuples (`p == (3, 4)`). Use `@dataclass` for mutability/validation/methods.
- `array.array` — typed, compact numbers: a **memory** win, not a speed win (elements are boxed on read). Fast maths → NumPy.

Key facts & numbers

List op	Cost
`len`, `a[i]`, `a[i] = x`, `append`, `pop()`	O(1) (append amortised)
`pop(i)`, `insert(i, x)`, `del a[i]`	O(n − i) → O(n) at front
`x in a`, `index`, `count`, `remove`	O(n)
`a[i:j]` / `extend(b)`	O(k)
`sort` / `sorted`	O(n log n); O(n) if sorted

	list	tuple	array.array	numpy.ndarray
Mutable	yes	no	yes	yes (fixed size)
Types	any	any	one C type	one dtype
Hashable	no	if items are	no	no
`+` means	concat	concat	concat	**element-wise add**

Need	Structure
Stack	list
Queue / both ends	`deque` (`list.pop(0)` is O(n))
Many membership tests	set / dict
Top-k	`heapq.nlargest(k, items)` — O(n log k)

Code to remember

```
grid = [[0] * cols for _ in range(rows)]   # independent rows
rows.sort(key=lambda r: (r.dept, -r.salary))
by_name = sorted(users, key=operator.attrgetter("name"))
first, *middle, last = [1, 2, 3, 4]       # middle == [2, 3]
lst = [v for v in lst if v != x]          # remove all x, O(n)
```

In the interview

Q: How is a list implemented? **A:** Dynamic array of pointers; O(1) index/append, O(n) insert/delete except at the end, O(n) membership.

Q: Linked list in Python? **A:** None built in; `deque` is a doubly-linked list of fixed-size blocks — O(1) at both ends, O(n) in the middle.

Q: Why is stability useful? **A:** Equal keys keep prior order, enabling multi-pass sorts.

Q: Shallow vs deep copy? **A:** Shallow duplicates only the outer container; deep copies everything recursively (slow, copies things you may want shared).

Q: Is `a, b = b, a` atomic? **A:** Build tuple then assign — no temp needed, but not thread-atomic.

Q: list vs tuple vs array vs NumPy? **A:** General mutable / immutable records and keys / compact numbers without deps / fast vectorised numerics.

Q: Comprehension vs loop? **A:** Comprehension for simple transforms (clearer, a bit faster); loop for multi-statement logic, try/except, side effects.

Gotchas

- `lst = lst.sort()` / `a = a.extend(b)` → `None`.
- `[[0]*3]*3` shares one row; `a[:]` doesn't copy nested lists.
- `def f(x=[])` → shared default; use `None`.
- Removing items while iterating → skips; iterate a copy or build new.
- `while x in lst: lst.remove(x)` → O(n²).
- `lst.clear()` empties the shared object; `lst = []` rebinds only this name.
- Sorting `"10" < "9"` as strings → `key=int`; mixed types → `TypeError`.
- `[print(x) for x in xs]` → useless list of `None`s.
- `x = 1,` makes a tuple; `(1,) + [2]` → `TypeError`.
- `array` `'l'` is 4 bytes on Windows, 8 on Linux/macOS → use `'q'` for 64-bit.
- Looping over a NumPy array in Python → lose the speed; vectorise.

TL;DR

List = dynamic array (O(1) end ops, O(n) elsewhere and for `in`); Timsort is stable; `=` aliases, `copy` shallow, `deepcopy` recursive; tuples for records/keys; `*rest` is a list.

6. Dictionaries, Sets and the `collections` Toolbox

Dicts, hashing, views, sets and set algebra, `Counter` / `defaultdict` / `deque` / `OrderedDict`, `heapq`, `bisect`, choosing a structure.

Must-know

- **dict** — hashable key → value; O(1) average get/set/delete/`in`; **insertion-ordered since 3.7** (equality ignores order). `d[k]` raises `KeyError`; `get` defaults; `in` checks **keys**.
- **Hash table** — open addressing: `hash(key)` picks a slot, collisions probe, **equality confirms**. Kept ≤ **2/3 full**, resizes. O(n) worst case with bad hashes.
- **Compact dict** — a hash index points into an entries array kept in insertion order (why dicts are ordered).
- **Hash contract** — equal objects must have equal hashes → `1`, `1.0`, `True` are the **same key**. Hash must never change.
- `__eq__` **without** `__hash__` → Python sets `__hash__ = None` (unhashable). Fix: hash the same immutable fields, or `@dataclass(frozen=True)`.
- **String hash salting** — per-process random salt (hash-flooding protection) → set order varies between runs.
- **Views** — `keys()/values()/items()` are live; key views support set ops; not indexable (`next(iter(d))`).
- **Merge** — `d1 | d2` new (right wins, 3.9+), `|=` / `update` in place, `{**a, **b}`.
- `setdefault(k, [])` — read-or-insert; `popitem()` removes the **last inserted** (LIFO).
- **set** — unique, unordered, hashable items; `remove` raises, `discard` doesn't; `set()` is empty (`{}` is a dict). **frozenset** — immutable, hashable (keys, sets of sets).
- **collections** — `Counter` (counts, `most_common`), `defaultdict` (auto-values), `deque` (O(1) both ends, `maxlen`), `OrderedDict` (`move_to_end`, `popitem(last=False)`), `ChainMap` (layered lookup), `namedtuple`.
- **heapq** — **min-heap** on a plain list; `h[0]` is the min; tuples `(priority, count, item)` break ties; max-heap = negate (or 3.14's `heappush_max` etc.).
- **bisect** — binary search on a **sorted** list; `key=` since 3.10; `insort` is O(n) (shifting).

Key facts & numbers

Operation	Cost
dict get/set/del/`in`, set `in`/add/remove	O(1) avg
`v in d.values()`	O(n)
`s | t`, `s ^ t`	O(len s + len t)
`s & t`	O(min(len s, len t)) — iterates the smaller
`s - t`, `s <= t`	O(len s)
`heapify` / push / pop / peek	O(n) / O(log n) / O(log n) / O(1)
`nlargest(k, it)`	O(n log k)
`bisect_left/right` / `insort`	O(log n) / O(n)

Need	Use
Count / top-k frequencies	`Counter.most_common(k)`
Group without key checks	`defaultdict(list)`
Queue, BFS, last-N buffer	`deque(maxlen=N)`
LRU cache	`OrderedDict` or `functools.lru_cache`
Dedupe keeping order	`list(dict.fromkeys(items))`
Config precedence	`ChainMap`
Thread-safe priority queue	`queue.PriorityQueue`

Hashable	Not hashable
int, float, str, bytes, None, frozenset, range, tuple of hashables, frozen dataclass, plain instances (by identity)	list, dict, set, bytearray, regular `eq=True` dataclass

Code to remember

```
class LRU:
    def __init__(self, cap): self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d: return -1
        self.d.move_to_end(k); return self.d[k]
    def put(self, k, v):
        self.d[k] = v; self.d.move_to_end(k)
        if len(self.d) > self.cap: self.d.popitem(last=False)
```

In the interview

Q: How does a dict work? **A:** Hash table with open addressing; hash → slot → probe on collision → `==` confirms; ≤ 2/3 full; O(1) avg, O(n) worst.

Q: Why can't lists be keys? **A:** Mutable → hash could change → entry lost.

Q: Implement an LRU cache. **A:** `OrderedDict` : `move_to_end` on access, `popitem(last=False)` on overflow — both O(1).

Q: Why deque for BFS? **A:** `popleft()` O(1) vs `list.pop(0)` O(n).

Q: `Counter` `+` vs `update`? **A:** `+` returns new and drops non-positive counts; `update` adds in place and keeps them.

Q: 3 most frequent words + first line each appears? **A:** Stream lines; `Counter` for counts; `setdefault(word, line_no)`; `most_common(3)` — O(total words).

Q: Why does set order change between runs? **A:** Salted string hashes.

Gotchas

- Resizing a dict while iterating → `RuntimeError`; iterate `list(d)`.
- `dict.fromkeys(keys, [])` → every key shares **one** list.
- `d.update(k=v)` uses the literal key `"k"`.
- Reading a missing key on a `defaultdict` **creates** it.
- `set("hello")` splits into characters; `{"hello"}` is one item.
- `s.union(t)` doesn't modify `s`; `s.update(t)` returns `None`.
- `set.pop()` is arbitrary, not the smallest.
- Float keys after arithmetic (`0.1 + 0.2`) are different keys.
- `OrderedDict` just for ordering on 3.7+ → a plain dict does it.
- `bisect` on an unsorted list → silently wrong.

TL;DR

Hash tables: O(1) avg, keys hashable, equal ⇒ same hash, dicts ordered (3.7+). Counter/defaultdict/deque/OrderedDict cover counting, grouping, queues, LRU; heapq for priority/top-k; bisect for sorted search.

7. Functions

Every kind of argument, LEGB scope, closures and late binding, first-class functions and `lambda`, decorators and `functools`, recursion, annotations.

Must-know

- `def` creates a function **object** bound to a name (has `__name__`, `__doc__`, `__defaults__`); calling creates a new frame. No `return` → `None`.

- **Defaults are evaluated once, at definition time** → a mutable default (`=[]` , `={}`) is shared across calls. Use `None` and create inside.
- `*args` → tuple of extra positionals; `**kwargs` → dict of extra keywords. At the call site `f(*seq)` , `f(**mapping)` unpack.
- **Positional-only** `/` (3.8, PEP 570) — params before `/` ; lets libraries rename them and allow the same key in `**kwargs` . **Keyword-only** `*` — params after `*` / `*args` ; use for boolean flags.
- `return a, b` returns **one tuple**.
- **LEGB** — Local → Enclosing → Global → Built-in. **Any assignment in a function makes the name local for the whole function** → `x += 1` on a global raises `UnboundLocalError` .
- `global` rebinds a module name; `nonlocal` rebinds the nearest enclosing *function* name (must exist). Mutating (`CACHE[k] = v` , `.append`) needs neither.
- **No block scope** — `if` / `for` don't create scopes; comprehensions and functions do. Class bodies are **not** an enclosing scope for methods.
- **Closure** — nested function that keeps outer variables alive (cells in `fn.__closure__` , names in `__code__.co_freevars`). Captures **variables, not values** → **late binding**.
- **First-class / higher-order** — functions can be passed, stored, returned. `map` / `filter` return **lazy one-shot** iterators.
- `lambda` — single expression; for tiny `key=` functions; don't assign one to a name (PEP 8) — use `def` .
- **Decorator** — `@d` over `def f` means `f = d(f)` . Wrapper `(*args, **kwargs)` + `@functools.wraps` (copies `__name__` / `__doc__` , sets `__wrapped__`) + `return result` . With arguments (`@retry(3)`) = three levels.
- **functools** — `partial` (pre-fill args, picklable), `reduce` (fold), `lru_cache(maxsize=N)` (bounded memo), `cache` (unbounded).
- **Recursion** — base case + shrinking input; one frame per level; limit ≈ **1000** (`RecursionError`); **no tail-call optimisation** (by design, keeps tracebacks).
- **Annotations** — not enforced at run time; read by mypy/pyright and frameworks (FastAPI/Pydantic); `inspect.get_annotations(f)` . 3.14 evaluates them lazily (PEP 649).

Key facts & numbers

Signature piece	Meaning
`def f(a, b, /, c, *, d)`	`a, b` positional-only; `c` either; `d` keyword-only
`def f(a=1, b)`	`SyntaxError` (non-default after default)
`f(a=1, 2)`	`SyntaxError` (positional after keyword)

Fibonacci	Time	Space
naive recursion	O(φⁿ) ≈ O(1.6ⁿ)	O(n) stack
`@cache` memoized	O(n)	O(n)

Caching caveat	Why
impure / time- or random-dependent functions	stale or wrong results
unhashable args	`TypeError`
`maxsize=None` on unbounded inputs	memory leak
instance methods	keys on `self` , keeps instances alive → use `cached_property`

Code to remember

```python
def retry(times):                    # decorator with arguments
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*args, **kwargs):
            for i in range(times):
                try: return fn(*args, **kwargs)
                except Exception:
                    if i == times - 1: raise
        return wrapper
    return deco

fns = [lambda i=i: i for i in range(3)]   # bind now; without i=i all return 2
```

In the interview

Q: Why is `def f(x=[])` dangerous? **A:** The list is created once at `def` time and shared by every call using the default; mutations accumulate.

Q: Explain LEGB and `UnboundLocalError`. **A:** Lookup goes L→E→G→B; assignment anywhere makes the name local everywhere in the function, so reading it before assignment fails.

Q: `global` vs `nonlocal`? **A:** Module scope vs nearest enclosing function scope.

Q: What's a closure; fix the loop-lambda bug? **A:** Inner function remembering outer variables; bind with `lambda i=i:` or `functools.partial`.

Q: How does a decorator work? **A:** Function in, function out; `wraps` preserves metadata; arguments need an extra outer layer.

Q: Does Python do TCO? **A:** No; depth ≈ 1000. Convert deep recursion to a loop with an explicit stack.

Q: Sort by two fields, one descending? **A:** Tuple key with a negated number, or two stable sorts (secondary first).

Q: Are type hints checked at run time? **A:** No — only by tools/libraries that read them.

Gotchas

- `result = cube` stores the function; `cube(3)` calls it. `key=len()` vs `key=len`.
- Printing instead of returning → caller gets `None`.
- Wrapper forgets `return result` → decorated function returns `None`.
- `@timed()` vs `@timed` mismatch for argument/no-argument decorators.
- Consuming a `map` / `filter` twice → second pass empty.
- `**kwargs` swallows typos (`timout=5`); prefer explicit keyword params.
- Recursion base `n == 1` loops forever on 0 → `n <= 1` ; forgetting `return` on the recursive call → `None`.
- `def walk(node, seen=set())` → shared accumulator.
- Shadowing built-ins: `list = [1, 2]; list("xy")` → not callable.

TL;DR

Defaults once (use `None`); `*args` tuple, `**kwargs` dict, `/` and `*` markers; LEGB + assignment-makes-local; closures bind late; decorators = `f = d(f)` + `wraps` ; memoize pure functions; no TCO, limit ~1000.

8. Modules, Packages and Environments

`import` internals, packages, `__main__` , circular imports, pip and venv, `pyproject.toml` , uv/Poetry, publishing.

Must-know

- **Module** — a `.py` file. `import m` binds `m` ; `from m import x` binds `x` ; **both load the whole module**.

- **Import steps** — check `sys.modules` cache → finders (`sys.meta_path` , then `sys.path`) → loader creates the module, **registers it in** `sys.modules` , runs top-level code **once** → bind the name.
- `sys.path` **order** — script dir, `PYTHONPATH` , stdlib, site-packages; **first match wins** (why `random.py` in your folder shadows the stdlib).
- **Regular package** — folder + `__init__.py` (runs on first import; re-exports the public API). **Namespace package** (PEP 420) — no `__init__.py` , can span folders.
- `import pkg` doesn't import submodules unless `__init__.py` does → `import pkg.sub` .
- `__all__` — names exported by `from m import *` ; documents the public surface.
- **Absolute imports** are PEP 8's default; **relative** (`from . import x`) only work inside packages, not in files run as scripts → `python -m pkg.mod` .
- `__name__` — `"__main__"` for the file being run, the module name when imported. `python -m pkg` runs `pkg/__main__.py` .
- **Circular import** — a module is in `sys.modules` before its body finishes, so the other side sees a half-built module; `from a import f` fails, `import a` survives (attribute looked up at call time).
- **venv** — isolated site-packages per project; in a venv `sys.prefix != sys.base_prefix` . conda also manages Python itself and native libraries.
- `requirements.txt` — pip input, usually exact pins (an environment). `pyproject.toml` — standard project file (PEP 517/518/621): metadata, dependency ranges, build system, tool config.
- **Libraries use ranges; applications lock** (lock file / pinned requirements). **uv** (fast, all-in-one) and **Poetry** manage pyproject + lock + venv.
- **Wheel** — pre-built `.whl` (a zip) that installs by unpacking; **sdist** — source that must be built.
- **src layout** — forces installing (`pip install -e .`) so tests run against the installed package.

Key facts & numbers

Error	Usual cause	Fix
`ModuleNotFoundError`	typo / installed in a **different interpreter** / wrong cwd	`python -m pip install` , check `sys.executable`
`ImportError: cannot import name`	name missing, or a **circular import**	restructure
`AttributeError: module 'x' has no attribute 'y'`	your `x.py` shadows the real module	rename, delete its `__pycache__`
`attempted relative import…`	ran a package module as a script	`python -m pkg.mod`

Tool	Isolates
venv	Python packages for an existing interpreter
pipx	each CLI tool in its own venv
conda	packages + native libs + Python itself
Docker	whole OS userland

`import math`	`from math import sqrt`
origin visible, no clashes	shorter, possible clashes
safer in cycles; sees monkey-patches	holds the old object when patched

Code to remember

```python
def main() -> None:
    ...
if __name__ == "__main__":    # keep only main() inside the guard
    main()
```

`python -m venv .venv && source .venv/bin/activate && python -m pip install -e .`

In the interview

Q: What happens on `import x`? **A:** `sys.modules` check → find on `sys.path` → create, register, execute once → bind. Later imports reuse the cached object.

Q: Does importing twice run the module twice? **A:** No — cached in `sys.modules` (use `importlib.reload` in a REPL).

Q: How do you fix a circular import? **A:** Move shared code to a third module, `import module` instead of names, or defer the import into a function.

Q: Installed but `ModuleNotFoundError`? **A:** pip used another interpreter/venv, or a local file shadows it.

Q: Is `__init__.py` required? **A:** No, but without it you get a namespace package; use it for normal packages.

Q: Library vs application dependencies? **A:** Ranges in the library's `pyproject.toml`; exact pins + lock file for apps.

Q: How do you publish? **A:** `python -m build` / `uv build` → sdist + wheel → TestPyPI → PyPI (twine/ `uv publish`, Trusted Publishing from CI). Versions are immutable.

Gotchas

- `import math.sqrt` → `import` names modules; use `from math import sqrt`.
- `sys.path.append` hacks → install the project (`-e .`) or run with `-m`.
- Slow work in `__init__.py` top level → slow imports and cycle risk.
- `sudo pip install`, committing `.venv/`, unpinned prod installs.
- `pip freeze` into a library's deps; exact pins in a library.
- PyPI name ≠ import name (`scikit-learn` / `sklearn`, `Pillow` / `PIL`).
- Test prints at module top level without the `__main__` guard run on every import.

TL;DR

`sys.modules` → `sys.path` → execute once; first match wins; cycles break `from x import y`. One venv per project, `python -m pip`; pyproject for projects, lock for apps; `python -m` for package modules.

9. Errors, Exceptions and Files

Exception hierarchy, `try` / `except` / `else` / `finally`, custom exceptions, chaining, EAFP, `ExceptionGroup`, `with`, file I/O, `pathlib`, csv/json/pickle.

Must-know

- **Exception** — an object raised on error; unwinds the stack until an `except` whose class matches (**subclasses included**); otherwise traceback and exit. Read tracebacks bottom-up: last line = type + message.
- **Hierarchy** — `BaseException` → `Exception` → your errors. `SystemExit`, `KeyboardInterrupt`, `GeneratorExit` sit directly under `BaseException` so `except Exception` doesn't swallow them.
- **try / except / else / finally** — first matching `except` wins (order specific → general); `else` runs only if nothing was raised (keeps `try` small); `finally` **always** runs, even when `try` returns.
- **Bare** `except:` also catches `KeyboardInterrupt` / `SystemExit` — never use it.
- `except E as e` — `e` is **deleted** at the end of the handler (breaks reference cycles).
- **Custom exceptions** — one library base class subclassing `Exception` + a few specific subclasses (optionally also a built-in: `class InvalidAmount(PaymentError, ValueError)`).
- **Bare** `raise` re-raises the active exception with its original traceback.
- **Chaining** — implicit (`__context__`, "During handling…"), explicit `raise X from e` (`__cause__`, "direct cause"), `from None` hides. `e.add_note()` (3.11+) adds context.
- **EAFP vs LBYL** — Python favours EAFP (try it, catch it): race-free, and `try` is almost free when nothing is raised (3.11+ "zero-cost"). LBYL when failure is common or partial work is hard to undo.
- `ExceptionGroup` + `except*` (3.11+) — bundle concurrent failures; each `except*` handles its subset; used by `asyncio.TaskGroup`.

- `assert` — internal invariants only; **stripped under** `python -O`; never for input validation.
- **Context manager** — `__enter__` / `__exit__(exc_type, exc, tb)`; a reusable try/finally; `__exit__` returning truthy **suppresses** the exception. `@contextmanager` = generator with `try/finally` around `yield`. `ExitStack` for a dynamic number (exits in reverse).
- **Files** — `with open(p, mode, encoding="utf-8")`; text = `str`, binary = `bytes`; iterate the file object for big files.
- **pickle is unsafe** — loading can call arbitrary callables (via `__reduce__`); never unpickle untrusted data.

Key facts & numbers

Mode	If exists	If missing
`r` (default)	read	`FileNotFoundError`
`w`	**truncates**	creates
`a`	append at end	creates
`x`	`FileExistsError`	creates
`+` / `b` / `t`	read+write / bytes / text (default)	

Format	Holds	Cross-language	Safe for untrusted
CSV (`newline=""`)	rows of strings	yes	yes
JSON	dict, list, str, int, float, bool, None	yes	yes
pickle	almost any Python object	no	**NO**

Task	pathlib
join / parts	`base / "a" / "b.txt"` , `.name .stem .suffix .parent`
read/write	`p.read_text()` , `p.write_bytes(b)`
mkdir / delete	`p.mkdir(parents=True, exist_ok=True)` , `p.unlink(missing_ok=True)`
search	`p.glob("*.py")` , `p.rglob("*.py")` , `p.walk()` (3.12+)

`shutil.copy/copytree/move/rmtree` (permanent), `tempfile.TemporaryDirectory()`; atomic write = temp file in the same folder + `os.replace`.

Code to remember

```python
class AppError(Exception): ...
class NotFound(AppError): ...

try:
    raw = path.read_text(encoding="utf-8")
except FileNotFoundError as e:
    raise NotFound(path) from e
else:
    data = json.loads(raw)

@contextlib.contextmanager
def timer():
    t = time.perf_counter()
    try: yield
    finally: print(time.perf_counter() - t)
```

In the interview

Q: When does `else` run; does `finally` run after `return`? **A:** `else` only if no exception; `finally` always, just before the function returns.

Q: Why isn't `KeyboardInterrupt` an `Exception`? **A:** So `except Exception` doesn't block Ctrl-C.

Q: `raise` vs `raise e`? **A:** Bare `raise` re-throws unchanged; `raise e` adds the current line.

Q: EAFP or LBYL? **A:** EAFP — avoids check-then-use races (file exists → open) and is cheap when errors are rare.

Q: How does `with` work? **A:** Calls `__enter__`, runs the block, always calls `__exit__` with exception info (all `None` on success).

Q: Why is `pickle.loads` dangerous? **A:** A pickle can instruct the loader to call any callable → code execution.

Q: JSON-encode a `datetime`? **A:** `default=` returning `isoformat()` or a custom `JSONEncoder`; it comes back as a string.

Q: `json.dump` vs `dumps`? **A:** File vs string.

Gotchas

- `except Exception as e: pass` → bugs vanish and resurface far away.
- `return` inside `finally` swallows exceptions (3.14 `SyntaxWarning`, PEP 765).
- Subclassing `BaseException` for app errors → escapes handlers.
- Returning error codes (`-1`) instead of raising.
- `assert (x > 0, "msg")` — a non-empty tuple is always true.
- `@contextmanager` without `try/finally` → no cleanup on error; `__exit__` returning `True` by accident swallows everything.
- `"w"` when you meant `"a"` → file wiped; no `encoding=` → breaks across OSes.
- `f.read()` on a multi-GB file → iterate lines.
- `csv` without `newline=""`; pickle in text mode; hard-coded `/tmp/x` names.

TL;DR

Specific `except`s, small `try`, `else` for success, `finally` / `with` for cleanup; custom hierarchy + `raise … from e`; EAFP; `assert` isn't validation; `open(..., encoding="utf-8")`, `pathlib`, JSON not pickle for untrusted data.

10. Object-Oriented Programming in Python

Classes, the data model (dunders), operator overloading, inheritance, MRO, mixins, ABCs and Protocols, composition, dataclasses, enums, `__slots__`, Pythonic patterns.

Must-know

- **Instance creation** — `__new__` creates, `__init__` initialises; `self` is explicit because a method is a plain function stored on the class (`obj.m()` passes `obj` first).
- **Attribute lookup** — instance `__dict__` first, then the class and its bases (MRO). Class attributes are **shared** (mutable ones are a classic bug).
- **Instance / class / static method** — `self` / `cls` (alternative constructors that work for subclasses) / neither.
- **Property** — start with a public attribute, upgrade to `@property` for validation without breaking callers; store the value in `_name` (same name → infinite recursion).
- **Data model** — syntax and built-ins call dunders: `len` → `__len__`, `+` → `__add__`, `for` → `__iter__`, `in` → `__contains__`, `obj()` → `__call__`.
- **`__repr__` vs `__str__`** — repr unambiguous for developers (used in containers); `str` falls back to `repr`, not vice versa → always write `__repr__`.
- **`__eq__` + `__hash__`** — must agree; defining `__eq__` alone makes the class **unhashable**; hash immutable fields.
- **`NotImplemented`** — return it (don't raise) for unsupported operands. `a + b` → `a.__add__(b)` → `b.__radd__(a)` → `TypeError`. `+=` tries `__iadd__` (must `return self`), then `+`.

- **`super()`** — calls the **next class in the MRO** of the instance's type, not "the parent"; always call `super().__init__()`.
- **MRO / C3 linearization** — children before parents, bases in declared order, consistent with every parent; `D(B, C)` with `B, C(A)` → D, B, C, A, object. Conflicting orders → class creation fails.
- **Mixin** — small, stateless, single-purpose class placed **left** of the main base.
- **Duck typing / ABC / Protocol** — behaviour over type / nominal, enforced at **instantiation** (`TypeError` if an `@abstractmethod` is missing), can share code / **structural**, checked by the type checker (`isinstance` only with `@runtime_checkable`).
- **Composition over inheritance** — has-a by default; inherit only for true is-a that satisfies Liskov (`class Stack(list)` leaks `insert(0, x)`).
- **`_x` / `__x`** — internal by convention / **name-mangled** to `_Class__x` to avoid subclass clashes; nothing is truly private.
- **`__slots__`** — fixed attributes, no per-instance `__dict__` → less memory, no accidental attributes; every class in the hierarchy must declare it; no weakrefs unless `"__weakref__"`; breaks `cached_property`.

Key facts & numbers

`@dataclass` flag	Effect
`eq=True` (default)	`__eq__` by field tuple → unhashable unless frozen
`order=True`	`< <= > >=` by field tuple in declaration order
`frozen=True`	assignment raises `FrozenInstanceError`; generates `__hash__` (shallow!)
`slots=True` / `kw_only=True` (3.10)	`__slots__` / keyword-only fields

Data holder	Mutable	Validation	Use when
dict	yes	none	dynamic keys, JSON
NamedTuple	no	none	immutable records that act like tuples
dataclass	configurable	`__post_init__`	default internal data
Pydantic	configurable	full parsing	untrusted input (APIs, config)

Enum kind	Note
`Enum`	own type; `Color.RED == 1` is False
`IntEnum` / `StrEnum` (3.11)	int/str interop (`HTTPStatus`)
`Flag`	combinable with `|`

Classic pattern	Pythonic
Strategy / Command	pass a function (`sorted(key=...)`), `partial`
Singleton	a module (cached in `sys.modules`)
Iterator	a generator
Factory	`@classmethod` constructor
Observer	list of callbacks

Code to remember

```python
@functools.total_ordering
@dataclass(frozen=True)
class Money:
    cents: int
    def __add__(self, other):
        if not isinstance(other, Money): return NotImplemented
        return Money(self.cents + other.cents)
    def __lt__(self, other): return self.cents < other.cents

@dataclass
class Order:
    items: list[str] = field(default_factory=list)
```

In the interview

Q: Why is `self` explicit? **A:** Methods are functions on the class; `obj.m()` passes `obj` — explicit is better than implicit.

Q: Classmethod or staticmethod for a factory? **A:** Classmethod, so `Sub.from_csv()` returns a `Sub`.

Q: What's `__new__` for? **A:** Controlling creation: subclassing immutables, instance caching, metaclasses.

Q: Why must equal objects have equal hashes? **A:** Hash picks the bucket, `==` confirms; mismatched hashes → duplicates in sets/dicts.

Q: Explain the MRO and `super()`. **A:** C3 linear order; `super()` = next in the MRO, so in a diamond each cooperating method runs once.

Q: ABC vs Protocol? **A:** ABC when I own the hierarchy and want enforcement/shared code; Protocol for structural typed interfaces accepting third-party types.

Q: Why does `lst += (1,)` work but `lst + (1,)` fail? **A:** `list.__iadd__` accepts any iterable (like `extend`); `__add__` only lists.

Q: Minimum to make a class iterable? **A:** `__iter__` returning an iterator, or `__getitem__` from 0 raising `IndexError`.

Gotchas

- Forgetting `self` → "takes 0 positional arguments but 1 was given".
- `self._next_id += 1` creates an instance attribute → `type(self)._next_id += 1`.
- Subclassing `dict` / `list`: C methods bypass overrides (`dict.update` skips your `__setitem__`) → `UserDict` / `UserList`.
- Forgetting `__rmul__` → `v * 2` works, `2 * v` fails.
- Mixing `super()` and `Parent.method(self)` → methods run twice or not at all.
- Forgetting to inherit `ABC` → `@abstractmethod` does nothing.
- Dataclass `tags: list = []` → `ValueError`; field without annotation is just a class attribute; no runtime type validation.
- Base field with default + subclass field without → `TypeError` (use `kw_only=True`).
- `member == "red"` on a plain `Enum` → always False.

TL;DR

Dunders make classes act like built-ins; `__repr__` first; `__eq__` / `__hash__` agree; `NotImplemented`; `super()` follows C3 MRO; composition by default; dataclasses for data (`default_factory`, `frozen`, `slots`); functions and modules replace much pattern boilerplate.

11. Iterators, Generators and Functional Tools

The iteration protocol, generators, `yield from`, lazy evaluation, `itertools`, comprehension style, `map` / `filter` / `reduce`.

Must-know

- **Iterable** — has `__iter__` returning a new iterator; can be looped many times (list, str, dict, range, set).
- **Iterator** — `__next__` returns items or raises `StopIteration`; its `__iter__` returns **itself**; **one-shot** (generator, file, `map`, `zip`, `enumerate`, `iter(list)`).
- `for` — calls `iter()` once, then `next()` until `StopIteration`.
- **Generator function** — contains `yield`; calling it returns a generator (an iterator) **without running the body**. Each `next()` runs to the next `yield` and suspends with locals intact.
- **Laziness** — constant memory, first result immediately, infinite sequences, composable pipelines; errors surface at consumption time.
- `return` **in a generator** ends iteration; the value goes to `StopIteration.value` (or is the result of `yield from`).
- **PEP 479 (3.7)** — `raise StopIteration` inside a generator becomes `RuntimeError`; use `return`.
- `yield from` delegates to a sub-iterable; `send` pushes a value in (prime with `next` first); `throw` raises inside; `close` triggers `GeneratorExit`.
- **Generator expression** `(expr for x in it)` — the lazy comprehension; pass directly to `sum`, `any`, `all`, `max`, `join`.
- `map` / `filter` / `zip` are lazy one-shot iterators in Python 3. Prefer comprehensions over `map(lambda…)`; built-ins over `reduce`.
- `any` / `all` short-circuit — give them generator expressions, not lists. `all([])` is `True`.
- `groupby` groups only **consecutive** equal keys → sort by the same key first.

Key facts & numbers

itertools	Does
`count`, `cycle`, `repeat`	infinite / repeating
`chain`, `chain.from_iterable`	concatenate / flatten one level
`islice(it, start, stop, step)`	slice any iterator (no negatives)
`takewhile`, `dropwhile`, `filterfalse`, `compress`	selection
`accumulate`	running totals/max
`pairwise` (3.10), `batched(it, n)` (3.12)	overlapping pairs / chunks
`zip_longest`, `tee`	pad / split (tee buffers in memory)

Combinatorics (n items, pick r)	Count	`"abc"`, r=2
`product(A, repeat=r)`	n^r	9
`permutations(it, r)`	$n!/(n-r)!$	6 (order matters)
`combinations(it, r)`	$n!/(r!(n-r)!)$	3
`combinations_with_replacement`	$(n+r-1)!/(r!(n-1)!)$	6

	List	Generator
Memory	all n	one item
Passes	many	one
`len`, index, slice	yes	no (`islice`)
Infinite	no	yes

Code to remember

```python
def read_records(path):
    with open(path, encoding="utf-8") as f:      # resource inside the generator
        for line in f:
            if line.strip():
                yield line.rstrip("\n")

class Bag:                                       # re-iterable class
    def __init__(self, items): self._items = list(items)
    def __iter__(self): yield from self._items

first_big = next((x for x in xs if x > 100), None)
```

In the interview

Q: Iterable vs iterator? **A:** Iterable gives a fresh iterator via `__iter__`; iterator has `__next__`, remembers position, is single-pass.

Q: Why is a generator empty the second time? **A:** It's an iterator already at its end; `list()` it or use a re-iterable class.

Q: Generator vs list comprehension? **A:** Lazy, one-shot, constant memory vs eager, reusable, indexable.

Q: How do you slice a generator? **A:** `itertools.islice` — generators have no `__getitem__`.

Q: Chunk into batches of 100? **A:** `itertools.batched(items, 100)` (3.12+); older `zip(*[iter(xs)]*100)` drops the remainder.

Q: Permutations vs combinations? **A:** Order matters (ab ≠ ba) vs doesn't.

Q: Why `itemgetter` over a lambda for `key=`? **A:** Slightly faster and self-describing.

Gotchas

- Looping twice over a generator/`map`/`zip`/file → second pass empty.
- `__iter__` returning `self` means the object is not re-iterable.
- Calling a generator function "does nothing" until iterated.
- Generators holding open files never finished → `with` inside, or `close()`/`contextlib.closing`.
- `groupby` on unsorted data → split groups.
- `any([expensive(x) for x in xs])` computes everything → drop the brackets.
- `reduce` on an empty sequence without an initial value → `TypeError`.
- `[x for x in xs if c else y]` → `SyntaxError`; choice `a if c else b` goes **before** `for`.
- Building a list only to take `[0]` → `next(genexp, default)`.
- Wrapping everything in `list(...)` "to be safe" loses laziness.

TL;DR

Iterables restart, iterators don't; `yield` makes lazy one-shot iterators; itertools before index-juggling (sort before `groupby`); comprehensions over `map`+lambda; genexps into `any`/`all`/`sum`.

12. Decorators, Context Managers and Metaprogramming

Decorators in depth, built-in decorators, context managers, descriptors, `__getattr__`, metaclasses and `__init_subclass__`, monkey patching, introspection.

Must-know

- **Decorator timing** — runs **once at definition** (usually import); the wrapper runs on every call. Stacked decorators **apply bottom-up, run top-down**.
- **Decorator factory** — `@retry(3)` = a function returning a decorator (three levels). `@retry` without `()` passes the function as `times`.
- **Class decorator** — takes and returns a class (e.g. `@dataclass`). Class-based decorators need `__get__` to work on methods.

- **Built-ins** — `@property`, `@classmethod`, `@staticmethod`, `@abstractmethod`, `@dataclass`, `@total_ordering`, `@lru_cache` / `@cache`, `@cached_property`, `@wraps`, `@contextmanager`, `@typing.override` (3.12).
- **Context manager** — `__enter__` return value binds to `as`; `__exit__(exc_type, exc, tb)` always runs; truthy return suppresses. In `@contextmanager`, an exception in the body is **thrown into the generator at** `yield`.
- **contextlib** — `suppress`, `closing`, `ExitStack` / `AsyncExitStack` (reverse unwind, even if one fails to open), `nullcontext`, `redirect_stdout`, `chdir` (3.11), `asynccontextmanager`.
- **Descriptor** — **class** attribute with `__get__` (+ `__set__` / `__delete__`). **Data descriptors** (have `__set__`) beat the instance dict; **non-data** ones don't. Power `property`, methods (function `__get__` → bound method), `classmethod`, `staticmethod`, `cached_property`, `__slots__`, ORM fields.
- `cached_property` is non-data → the instance dict wins after the first computation.
- `__getattr__` — called only when normal lookup **fails** (proxies, lazy attrs). `__getattribute__` — on **every** access (rare; recursion-prone → use `super().__getattribute__`). Module `__getattr__` (PEP 562) for lazy imports.
- **Dunders bypass both hooks** — `len(p)`, `p + 1` look up on the type; a proxy must define them explicitly.
- **Metaclass** — the class of a class (`type`, and `type(type) is type`); runs when a `class` statement executes. Used by Django ORM, `abc`. Conflicts: bases with unrelated metaclasses → `TypeError`.
- **Preference order** — class decorator → descriptor → `__init_subclass__` (registries, validation) → metaclass.
- **Monkey patching** — runtime modification; OK in tests via `mock.patch` / `monkeypatch` (auto-restore, **patch where the name is looked up**); in prod prefer dependency injection.
- **Introspection** — `getattr/setattr/hasattr`, `vars`, `dir`, `inspect.signature` (FastAPI/pytest inject by signature). `eval` / `exec` never on untrusted input → `ast.literal_eval` for data.

Key facts & numbers

Need	Tool
modify one class	class decorator
reusable attribute behaviour	descriptor
register/validate every subclass	`__init_subclass__`
control class namespace / `isinstance` / DSL	metaclass

	Class-based CM	`@contextmanager`
Size	more code	less
Reusable/stateful	yes	one-shot per call
Errors	`__exit__` args	`try/except/finally` around `yield`
As decorator	via `ContextDecorator`	yes

Code to remember

```python
def timed(fn):
    @functools.wraps(fn)
    def wrapper(*args, **kwargs):
        t = time.perf_counter()
        try: return fn(*args, **kwargs)
        finally: print(fn.__name__, time.perf_counter() - t)
    return wrapper

class Positive:                    # data descriptor
    def __set_name__(self, owner, name): self.name = "_" + name
    def __get__(self, obj, objtype=None):
        return self if obj is None else getattr(obj, self.name)
    def __set__(self, obj, value):
        if value <= 0: raise ValueError(value)
        setattr(obj, self.name, value)
```

In the interview

Q: When does a decorator run? **A:** Once at definition; the wrapper on each call.

Q: Order of stacked decorators? **A:** Applied bottom-up, executed top-down — e.g. caching outside an auth check would cache across users.

Q: How does `with` handle exceptions? **A:** `__exit__` gets the exception; truthy return suppresses; otherwise it propagates.

Q: What is a descriptor? **A:** A class attribute intercepting get/set/delete; data descriptors outrank the instance dict — how properties and methods work.

Q: `__getattr__` vs `__getattribute__`? **A:** Fallback on failure vs every access.

Q: What's a metaclass; do you use one? **A:** Class of a class; rarely — decorators, descriptors and `__init_subclass__` cover most needs without conflicts.

Q: Safe alternative to `eval` for data? **A:** `ast.literal_eval`.

Gotchas

- Wrapper without `return` → `None`; missing `@wraps` → wrong name/docstring.
- Heavy work at decoration time runs on import.
- `@contextmanager` yielding twice → `RuntimeError: generator didn't stop`.
- Using a file after its `with` block → `ValueError: I/O operation on closed file`.
- Descriptor storing values on itself → shared by all instances; descriptors in `__init__` don't work; handle `obj is None`.
- `__getattr__` returning `None` for unknown names → breaks `hasattr`, copy, pickle; raise `AttributeError`.
- Forgetting `super().__init_subclass__(**kwargs)`.
- Patching the definition site instead of where the name is used.

TL;DR

Decorators = `f = d(f)` at definition, `wraps`, factories for args; `with` = enter/body/exit (truthy suppresses); descriptors power properties/methods; `__getattr__` for fallbacks; metaclasses last.

13. Type Hints and Static Typing

Annotations, generics, `Optional`/unions, `Any`, `TypeVar`, `Callable`, `Protocol`, `TypedDict`, `Literal` / `Final` / `NewType`, mypy/pyright, pydantic.

Must-know

- **Type hints** (PEP 484) — optional, **ignored at run time** (`x: int = "a"` runs fine); a static checker (**mypy**, **pyright**) verifies them. **Gradual**: adopt incrementally. No speed-up (mypyc is a separate compiler).

- **Syntax** — `param: T`, `-> T`, `var: T = value`; annotate signatures, let inference handle locals. Hints live in `__annotations__`; read with `get_type_hints`.
- **Built-in generics** (3.9) — `list[int]`, `dict[str, float]`, `tuple[int, str]` (exactly two), `tuple[int, ...]` (any length), `type[User]` (the class).
- **Parameters: accept abstract** — `Iterable`, `Sequence`, `Mapping` from `collections.abc`. **Mutable containers are invariant**: `list[int]` is not a `list[float]` (callee could insert a float).
- `X | None` (3.10) = `Optional[X]` — "may be None", not "has a default". Narrow with `is not None`, `isinstance`, early return.
- `Any` **vs** `object` — `Any` switches checking off and is **contagious**; `object` accepts everything but forces narrowing. Contain `Any` at boundaries.
- **TypeVar / generics** — tie input to output types; 3.12 syntax `def first[T](xs: Sequence[T]) -> T`, `class Stack[T]` (scoped, variance inferred). **Bound** = any subtype of X; **constrained** = exactly one of the listed types.
- `Callable[[int, str], bool]` types function values; `ParamSpec` lets decorators preserve the wrapped signature.
- **Protocol** — structural "duck type" checked statically; keep it small (1–3 methods); `@runtime_checkable` `isinstance` checks **method names only**.
- **TypedDict** — types dict-shaped data (JSON, `**kwargs` via `Unpack`); a plain dict at run time.
- `Literal` fixed values, `Final` no reassignment, `NewType` distinct type at zero runtime cost (vs alias `UserId = int`, interchangeable). Plus `Annotated`, `Self` (3.11), `Never` / `NoReturn`, `ClassVar`, `@overload`, `TypeIs` (3.13), `LiteralString`.
- **Type alias** — `type Name = ...` (3.12+); readability, not safety.
- **Static vs runtime** — static checks verify **code**; runtime validation verifies **data**. Validate untrusted input once at the boundary with **pydantic** ("parse, don't validate"); its v2 core is Rust.

Key facts & numbers

mypy error	Fix
`[union-attr]` Item "None" has no attribute	narrow: `if x is not None` / early return
`[var-annotated]` Need type annotation	`d: dict[str, int] = {}`
`[arg-type]` / `[assignment]`	convert, fix signature, new variable
`[return]` Missing return	return/raise in every branch
`[import-untyped]`	install stubs (`types-requests`)
`[no-any-return]`	validate, then `cast`

Runtime behaviour	
annotations, dataclass fields, `NewType`, `cast`, `Final`, `TypedDict`	never checked
`isinstance(x, int | str)`	works
`isinstance(x, list[int])`	`TypeError`

Protocol	ABC
structural, checker-enforced	nominal, enforced at instantiation
works with third-party classes	needs inheritance or `register()`
describe a shape	share code + force overrides

Code to remember

```python
from collections.abc import Iterable
from typing import Protocol

class SupportsClose(Protocol):
    def close(self) -> None: ...

def close_all(items: Iterable[SupportsClose]) -> None:
    for it in items: it.close()

def first[T](xs: list[T]) -> T | None:
    return xs[0] if xs else None
```

In the interview

Q: Are hints enforced? **A:** No — stored only; checkers and libraries (pydantic, FastAPI) read them.

Q: Would mypy catch a bad API payload? **A:** No — it never sees runtime data; validate at the boundary.

Q: What does a TypeVar buy you? **A:** `first(list_of_ints)` returns `int`, not `Any`.

Q: Why isn't `list[int]` a `list[float]`? **A:** Invariance — a callee could append a float.

Q: Protocol or ABC? **A:** ABC to share code and force implementations; Protocol to describe a shape.

Q: How do you run checking? **A:** mypy/pyright in CI + pre-commit, strict for new code, per-module overrides for legacy; `reveal_type` to debug; `# type: ignore[code]` only with a code.

Gotchas

- `def f(x: str = None)` → must be `str | None` (implicit Optional is off in mypy).
- `tuple[int]` is a one-element tuple → `tuple[int, ...]`.
- Bare `list` = `list[Any]`.
- mypy skips bodies of un-annotated functions (unless `check_untyped_defs` / `--strict`); add `-> None`.
- `cast` is not validation.
- `if x:` to narrow when 0/"" are valid → `is not None`.
- Returning `X | None` "just in case" pushes checks onto every caller → raise instead.
- TypeVar used once in a signature adds nothing → `object`.

TL;DR

Hints are static, unenforced, gradual; accept abstract types; `X | None` must be narrowed; `Any` leaks, `object` doesn't; generics tie types; Protocols for shapes; pydantic at the edges, mypy inside.

14. A Tour of the Standard Library

`datetime` / `zoneinfo`, `time`, `re`, `logging`, `argparse`, `os` / `sys` / `subprocess`, `random` vs `secrets`, `math` / `statistics` / `decimal`, small modules, `sqlite3`, `urllib`, when to go third-party.

Must-know

- **Datetimes** — keep them **aware and in UTC** inside, convert to a `ZoneInfo` zone only for display, serialise as **ISO 8601**. Naive and aware can't be compared. `datetime.now(UTC)`, not `utcnow()` (naive). `timedelta` has **no months**. `zoneinfo` replaces `pytz`.
- **Clocks** — `time.time()` = "when" (can jump); `perf_counter()` = "how long"; `monotonic()` = timeouts. Micro-benchmarks → `timeit`.
- **Regex extras** — named groups, `re.VERBOSE`, lookahead/lookbehind, `finditer` for large input, `fullmatch` for validation; 3.11 atomic groups `(?>…)` and possessive `*+` prevent backtracking. Don't regex JSON/HTML/CSV/URLs.
- **logging** — **logger** (per module, `getLogger(__name__)`, dotted hierarchy) → **handler** (where) → **formatter** (how) (+ filters). The **app** configures once (`dictConfig`); **libraries never configure**. Lazy `%s` args, `log.exception` in `except`, structured JSON to stdout in containers. Default root level is **WARNING**.

- **argparse** — positionals + options with `type` / `choices` / `action`; subparsers; `main(argv=None)` for testability; exit code 2 on usage errors. `type=bool` is wrong → `action="store_true"`.
- **subprocess** — `subprocess.run([...], check=True, capture_output=True, text=True, timeout=...)` with a **list**; never `shell=True` on user input.
- **`os`** OS + env (`os.environ.get`, `os.replace` atomic rename), **`sys`** interpreter (`argv`, `path`, `executable`, `modules`), **`shutil`** trees, **`pathlib`** paths.
- **`random` vs `secrets`** — reproducible simulation vs anything an attacker must not guess; compare secrets with `secrets.compare_digest` (timing-safe).
- **math/statistics** — `isclose` (rel tol 1e-9), `isqrt`, `comb`, `perm`, `gcd`, `lcm` (3.9), `fsum`; `round` is banker's rounding; `stdev` (sample, n−1) vs `pstdev` (population).
- **Hashing** — SHA-256 for integrity; scrypt/pbkdf2/argon2 for passwords. **UUIDs** — `uuid4` default; `uuid7` (3.14) time-ordered (compact DB indexes); avoid `uuid1` (leaks MAC).
- **sqlite3** — `connect` → `execute` with `?` placeholders → `with con:` transaction → `Row`. One writer at a time.
- **urllib** — `urllib.parse` great; for real clients use **requests/httpx** (pooling, JSON, retries, async); always `timeout=`.

Key facts & numbers

Expression	Result
`math.floor(-2.5)` / `int(-2.5)`	`-3` / `-2`
`round(2.5)`, `round(3.5)`	`2`, `4`
`math.comb(5, 2)`	`10`

Need	stdlib → third-party
HTTP client	urllib.request → requests, httpx
CLI	argparse → click, typer
Validation	dataclasses → pydantic
Tables	csv → pandas, polars
Testing	unittest → pytest
DB	sqlite3 → SQLAlchemy, psycopg
Passwords	hashlib.scrypt → argon2-cffi, bcrypt

Handy small modules: `tomllib` (3.11, read-only TOML), `shlex.split`, `difflib.get_close_matches`, `textwrap.dedent`, `base64`, `gzip.open(..., "rt")`, `getpass`, `pprint.pp`.

Code to remember

```
log = logging.getLogger(__name__)
try:
    r = subprocess.run(["git", "status"], capture_output=True,
                       text=True, check=True, timeout=10)
except subprocess.CalledProcessError:
    log.exception("git failed")
token = secrets.token_urlsafe(32)
con.execute("SELECT * FROM users WHERE name = ?", (name,))
```

In the interview

Q: How do you handle time zones? **A:** Aware UTC internally, `ZoneInfo` at display, ISO 8601 on the wire; never hard-coded offsets.

Q: Why not measure with `time.time()`? **A:** NTP adjustments can make durations wrong or negative; use `perf_counter`.

Q: How do you set up logging? **A:** `getLogger(__name__)` per module; app configures once with `dictConfig`; libraries don't.

Q: Why not `print`? **A:** No levels, routing, timestamps, per-module control.

Q: Debug logs missing? **A:** Logger/handler level above DEBUG (root defaults to WARNING).

Q: How do you run a shell command safely? **A:** `subprocess.run` with an argument list, `check=True`, timeout, no `shell=True` on input.

Q: When not SQLite? **A:** Many concurrent writers or a DB shared across servers → PostgreSQL/MySQL.

Gotchas

- `basicConfig` in an imported module; handlers added in a repeatedly called function → duplicate lines.
- Duplicate logs from child + root handlers with `propagate=True`; `dictConfig` `disable_existing_loggers` defaults to True.
- `parse_args()` at import time breaks tests.
- `os.chdir` in library code changes the whole process.
- SQL built with f-strings → injection; forgetting `commit`; `(cat)` instead of `(cat,)`.
- `findall` with groups returns only the groups.
- No `timeout=` on HTTP → hangs forever; hand-built query strings → use `urlencode`.

TL;DR

UTC-aware datetimes, `perf_counter` for timing, logger-per-module configured once, `subprocess.run([...], check=True)`, `secrets` for tokens, `?` placeholders, stdlib first then requests/pytest/pydantic/pandas.

15. Testing and Debugging

`unittest`, pytest (fixtures, parametrize, marks), test doubles and mocks, coverage, doctest, TDD, Hypothesis, tracebacks, pdb, logging vs print.

Must-know

- **Test pyramid** — many fast, isolated, deterministic **unit** tests; fewer **integration** (real DB/files) and **E2E** tests; all in CI. Shape: **Arrange–Act–Assert**. Test behaviour, not implementation.
- **unittest** — built in; `TestCase` classes, `test_` methods, `setUp` / `tearDown`, `assertEqual` / `assertRaises`, `subTest`; `python -m unittest`.
- **pytest** — plain functions + `assert` (rewritten to show values/diffs); runs unittest tests unchanged.
- **Fixtures** — reusable setup injected by parameter name; `yield` for teardown; scope default **function** (isolation), `session` for expensive read-only resources; shared ones in **`conftest.py`**.
- **Built-in fixtures** — `tmp_path`, `monkeypatch`, `capsys`, `caplog`.
- **`@pytest.mark.parametrize`** — data-driven cases. **skip** = don't run; **xfail** = run expecting failure (`strict=True` flags an unexpected pass).
- **Doubles** — dummy (unused), **stub** (canned answers), spy (records calls), **mock** (verifies calls), **fake** (working simplified implementation, e.g. in-memory repo).
- **`unittest.mock`** — `return_value` to stub, `side_effect` for errors/sequences, `assert_called_once_with` only when the interaction matters. **`autospec`** / **`spec`** so mocks fail on non-existent methods/signatures.
- **`patch`** **where the name is looked up**, not where it's defined. Prefer **dependency injection** + fakes; mock only at boundaries.
- **Coverage** — shows what is **not** tested, not that code is correct; `pytest --cov=pkg --cov-branch --cov-report=term-missing`; thresholds via `--cov-fail-under`. **Mutation testing** (`mutmut`) checks tests actually fail on changed code.
- **doctest** — keeps docs examples correct; brittle for real suites.
- **TDD** — red → green → refactor; at minimum, **reproduce every bug with a failing test first**.
- **Hypothesis** — state properties, it generates and **shrinks** counter-examples (parsers, serializers, invariants).

- **Tracebacks** — read bottom-up. **pdb** via `breakpoint()`; `python -m pdb`, `pdb.pm()` post-mortem, `pytest --pdb`, `--trace`.

Key facts & numbers

pytest flag	Does
`-k "expr"` / `-m "not slow"`	select by name / mark
`-x`, `--maxfail=2`	stop early
`--lf`	rerun last failed
`-s`	show prints
`--pdb`	debugger on failure
`--durations=5`	slowest tests

pdb	
`n` / `s` / `r` / `c`	next (over) / step (into) / return / continue
`p`, `pp`, `w`, `u`, `d`	print, pretty-print, stack, up, down
`b file:line, cond` / `unt N`	conditional breakpoint / run until line
`interact`	REPL in current scope

Error	Usual cause
`'NoneType' object has no attribute`	function returned `None` (missing `return`, `list.sort()`)
`takes 2 positional arguments but 3 were given`	forgot `self`
`UnboundLocalError`	assignment makes the name local
`ModuleNotFoundError`	wrong venv or shadowing file

Code to remember

```python
@pytest.fixture
def repo(tmp_path):
    r = FileRepo(tmp_path / "db.json")
    yield r
    r.close()

@pytest.mark.parametrize("price, pct, expected", [(100, 10, 90), (50, 0, 50)])
def test_discount(price, pct, expected):
    assert apply_discount(price, pct) == pytest.approx(expected)

def test_rate():
    with patch("shop.orders.get_rate", autospec=True, return_value=83.5) as m:
        convert(10, "INR"); m.assert_called_once_with("INR")
```

In the interview

Q: unittest vs pytest? **A:** pytest: plain asserts with rich diffs, composable scoped fixtures, parametrize, plugins; it also runs unittest tests.

Q: Mock vs stub? **A:** A stub supplies answers; a mock also verifies how it was called.

Q: Why autospec? **A:** So the mock rejects methods/signatures the real object doesn't have.

Q: Where do you patch? **A:** Where the name is looked up (`shop.orders.get_rate`, not `rates.get_rate`).

Q: Is 100% coverage the goal? **A:** No — it shows untested lines, not correctness; target business logic and error paths.

Q: How do you debug? **A:** Reproduce, hypothesise, observe (`breakpoint()`, logs), narrow, fix, keep the regression test.

Q: print or logging? **A:** print for a quick look then delete; logging for anything that stays (`caplog` to test it).

Gotchas

- Tests depending on order or shared module state → fresh fixtures.
- Float asserts with `==` → `pytest.approx`.
- `pytest.raises` wrapping too much code.
- Same-named test files in different folders without `__init__.py` → import mismatch.
- Plain `Mock()` hides typos; misspelled asserts (`assret_called`) now raise `AttributeError`.
- Tests that only prove the mock returns what you told it.
- Committed `breakpoint()` hangs CI → `PYTHONBREAKPOINT=0`, lint rule `T100`.
- Flaky tests from unseeded randomness or the real clock → seed, inject the clock.

TL;DR

pytest + fixtures (`yield`, `conftest.py`) + parametrize; fakes via DI, mocks with autospec patched where used; coverage finds gaps, not quality; reproduce bugs with a failing test; `breakpoint()` + `n/s/c/p/w`.

16. Concurrency and Async

Threads, locks, queues, the GIL and free-threading, multiprocessing, executors, asyncio, choosing a tool.

Must-know

- **Concurrency vs parallelism** — overlap **waiting** (threads, asyncio; one core is enough) vs overlap **computing** (multiprocessing, free-threaded threads, C code releasing the GIL). Measure first: I/O-bound vs CPU-bound.
- **Process** — own memory. **Thread** — shares its process's memory. **Task** — coroutine on an event loop; many per thread.
- **Race condition** — outcome depends on timing, usually an unprotected read-modify-write. **`x += 1` is several bytecodes → not atomic**, GIL or not.
- **GIL** — CPython mutex: one thread runs bytecode at a time. Exists because refcounting isn't thread-safe without it; keeps single-threaded code and C extensions simple. **Released during blocking I/O** and by many C extensions (numpy). Protects the interpreter's internals, **not your logic**.
- **Free-threading** — optional build: experimental in 3.13, **officially supported but not default in 3.14** (`3.14t`); check C-extension support; single-threaded code gets a bit slower. PyPy has a GIL; Jython/IronPython don't.
- **Lock** (not re-entrant) vs **RLock** (same thread may re-acquire); `Semaphore(n)`, `Event`, `Condition`, `Barrier`. Use `with lock:`, keep critical sections tiny, never hold during slow I/O.
- **`queue.Queue`** — thread-safe FIFO for producer–consumer; bound it for backpressure; stop with sentinels or `shutdown()`. Share a queue, not variables.
- **multiprocessing** — real CPU parallelism; each process has its own GIL; arguments/results are **pickled**. **Start methods**: `spawn` (Windows, macOS), `forkserver` (Linux default from **3.14**), `fork` (unsafe with threads). Needs the **`__main__` guard** and top-level functions.
- **`concurrent.futures`** — one API for `ThreadPoolExecutor` / `ProcessPoolExecutor`; `submit` → Future; **`result()` returns or re-raises** the worker's exception (never checking = silently lost); `map` keeps order; `as_completed` yields in finish order; `timeout=`.
- **asyncio** — single-threaded **cooperative** concurrency; tasks switch **only at `await`**; thousands of connections cheaply; **anything blocking stalls everything** → async libraries or `asyncio.to_thread`.
- **`TaskGroup`** (structured concurrency) — on failure **cancels siblings** and raises an `ExceptionGroup` (`except*`); **`gather`** leaves siblings running (or `return_exceptions=True`).

- **Deadlock conditions** — mutual exclusion, hold-and-wait, no pre-emption, circular wait; lock ordering breaks circular wait. Find hung threads with `faulthandler` or `py-spy dump`.

Key facts & numbers

Situation	Use
tens–hundreds of blocking I/O calls, sync libs	`ThreadPoolExecutor`
thousands of connections, async libs	`asyncio`
CPU-bound pure Python	`ProcessPoolExecutor`
CPU-bound numeric	numpy/pandas (vectorise)
async app + sync lib / heavy CPU	`to_thread` / `run_in_executor(ProcessPoolExecutor())`
beyond one machine	Celery, RQ, Ray, Dask, Spark

	Threads	Processes	asyncio
Parallel CPU (GIL build)	no	yes	no
Memory per unit	~MB stack	tens of MB	~KB
Shared memory	yes (locks)	no (pickle/IPC)	yes (one thread)
Switching	pre-emptive	OS	cooperative at `await`
Race risk	high	low	low

Code to remember

```
lock = threading.Lock()
def deposit(n):
    global balance
    with lock: balance += n

with ThreadPoolExecutor(max_workers=8) as ex:
    for fut in as_completed([ex.submit(fetch, u) for u in urls]):
        print(fut.result())          # re-raises worker errors

async def main():
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(fetch(u)) for u in urls]
    return [t.result() for t in tasks]
asyncio.run(main())
```

In the interview

Q: What is the GIL and why does it exist? **A:** One-thread-at-a-time bytecode lock protecting CPython's refcounting; released on I/O, so threads suit I/O-bound work; CPU-bound → processes or numpy.

Q: Does the GIL make `x += 1` atomic? **A:** No — a switch can happen between load and store.

Q: Is `list.append` thread-safe? **A:** A single append on a built-in list is, in CPython; sequences of operations are not.

Q: Why doesn't numpy suffer from the GIL? **A:** Its loops run in C and release it.

Q: Threads vs processes vs asyncio? **A:** Sync I/O / CPU parallelism / massive I/O concurrency with async libraries.

Q: Is asyncio parallel? **A:** No — one thread, concurrency only.

Q: `gather` vs `TaskGroup`? **A:** TaskGroup cancels siblings on error and raises an `ExceptionGroup`.

Q: Worker exception in an executor? **A:** Stored in the Future; raised on `result()`.

Gotchas

- "The GIL makes my code thread-safe" → check-then-set can still run twice.
- Unhandled exception in a raw `Thread` → printed, thread dies silently; use executors or `threading.excepthook`.
- Missing `__main__` guard with `spawn` → `RuntimeError`/runaway spawning; lambdas to processes → `PicklingError`.
- Child modifying a global doesn't change the parent.
- `fork` after threads started → random hangs.
- Tiny tasks in processes → pickling/startup dominates.
- Forgotten `await` ("never awaited"); dropped `create_task` reference → task may be GC'd.
- `asyncio.run()` inside a running loop (Jupyter) → `RuntimeError`.
- Swallowing `CancelledError` breaks timeouts; asyncio objects aren't thread-safe.

TL;DR

I/O → threads or asyncio; CPU → processes or numpy; GIL limits parallel bytecode, not I/O, and never makes your logic safe; locks/queues for shared state; executors + `result()`; `TaskGroup`, never block the loop.

17. Performance, Memory and CPython Internals

The object model, refcounting and GC, int caching and interning, big-O of built-ins, profiling, optimisation techniques, recent CPython speed-ups.

Must-know

- **PyObject** — every value is a heap object whose header holds a **reference count** and a **type pointer**. Identity (`id`, the address in CPython — may be **reused** after the object dies), type (fixed), value (mutable or not).
- **Reference counting** — frees an object **immediately** when its count hits zero. **Immortal objects** (3.12+) skip counting.
- **Cyclic GC** — generational collector that periodically finds unreachable **cycles** of container objects; controls `gc.collect/disable/freeze/set_threshold`.
- `weakref` — refer without owning (caches, back-pointers); `weakref.finalize` over `__del__` for cleanup.
- **Leaks** happen — globals/caches that grow, lists never cleared, stored exceptions keeping frames alive. Find with `tracemalloc` snapshots, `gc.get_referrers`, `objgraph`.
- **Small-int cache (−5…256) and string interning** — an optimisation, not a rule; compare with `==`.
- **Memory** — a list of 1M ints ≈ 36 MB (8-byte pointers + ~28-byte int objects); a numpy array ≈ 8 MB (raw contiguous values).
- **Profile before optimising** — `timeit` (snippets; take the min of repeats), `cProfile` (functions; adds per-call overhead), `line_profiler` (lines), **sampling** `py-spy` / `scalene` / `pyinstrument` (low overhead, live processes), `tracemalloc` (allocations).
- **Optimisation order** — measure → algorithm/data structure → cache → vectorise → stream → micro-tweaks → native code / parallelism → upgrade Python.
- **Why numpy is fast** — contiguous typed memory, one compiled loop, no per-element objects/dispatch, SIMD, BLAS.
- **Native routes** — numba (`@njit`), Cython, Rust + PyO3/maturin (pydantic-core, polars, ruff), C/C++ + pybind11, PyPy (tracing JIT), mypyc.

Key facts & numbers

Operation	list	dict / set	deque
index / key lookup	O(1)	O(1) avg	O(1) ends, O(n) middle
`x in c`	**O(n)**	O(1) avg	O(n)
append right / pop right	O(1) amortised	O(1)	O(1)
insert/pop left	**O(n)**	—	**O(1)**
delete middle	O(n)	O(1) avg	O(n)
sort	O(n log n)	—	—

Technique	Typical gain
better algorithm / structure	10×–100,000×
caching (`lru_cache`)	exponential → linear
numpy/pandas vectorisation	10×–100×
generators	memory GB → KB
built-ins, comprehensions	1.1×–3×
local aliases in hot loops	~1.1× on 3.11+
`__slots__` (millions of objects)	less memory per object

Version	Speed/internals change
3.11	specialising adaptive interpreter (PEP 659), zero-cost exceptions; **10–60 % faster, ~1.25× avg**
3.12	inlined comprehensions (PEP 709), immortal objects (PEP 683), per-interpreter GIL (PEP 684)
3.13	experimental JIT (off by default), experimental free-threading, new REPL
3.14	free-threading officially supported (PEP 779), optional tail-calling interpreter

Code to remember

```
seen = set(big_list)                 # O(n) once …
hits = [x for x in queries if x in seen]   # … then O(1) per lookup

import tracemalloc
tracemalloc.start(); s1 = tracemalloc.take_snapshot()
work(); s2 = tracemalloc.take_snapshot()
for stat in s2.compare_to(s1, "lineno")[:5]: print(stat)
```

`python -m cProfile -s cumtime app.py` · `python -m timeit "sum(range(1000))"` · `py-spy top --pid 1234`

In the interview

Q: How does CPython manage memory? **A:** Refcounting frees immediately; a generational cyclic GC handles cycles; weakrefs avoid ownership.

Q: Can Python leak memory? **A:** Yes — anything still referenced (caches, growing lists, cycles held by globals).

Q: Why is `append` amortised O(1)? **A:** Over-allocation; rare full copies averaged over many appends.

Q: When is dict lookup O(n)? **A:** Pathological hash collisions.

Q: How do you speed up slow Python? **A:** Profile, fix the algorithm (set lookups, deque, heapq), cache, vectorise, then native code or parallelism.

Q: Downside of `lru_cache`? **A:** Memory growth, stale results, hashable args only.

Q: What made 3.11 faster? **A:** Specialising adaptive interpreter, cheaper frames, inlined Python calls, zero-cost exceptions.

Gotchas

- `is` for value comparison; treating `id()` as unique forever.
- `__del__` for important cleanup → `with` or `weakref.finalize`.
- `gc.collect()` in a loop doesn't fix references you still hold.
- `x in list` inside a loop → O(n²); but building a set for a *single* lookup isn't worth it.
- Single-run timings, tiny inputs, trusting cProfile on many tiny functions.
- Calling numpy element-by-element in a Python loop (slower than plain Python).
- Unbounded caches in long-running services; premature `__slots__`.

TL;DR

Everything is a refcounted object; GC collects cycles; small ints/strings are cached (use `==`); know the big-O table; profile first, fix the algorithm, then cache/vectorise/stream; upgrade Python.

18. The Data Stack: numpy, pandas, matplotlib

Arrays and broadcasting, linear algebra, pandas selection/cleaning/groupby/merge/reshape/dates, performance, plotting.

Must-know

- **ndarray** — one dtype, one contiguous block, a shape tuple; whole-array operations run as C loops (tens of times faster than Python loops).
- **Views vs copies** — basic slicing returns a **view** (shares memory); boolean masks and fancy (int-list) indexing return **copies**.
- **Fixed-size dtypes overflow** — `np.int8(127) + np.int8(1)` → −128 (arrays wrap silently); Python ints never overflow.
- **Broadcasting** — align shapes **from the right**; dimensions must be equal or 1; size 1 stretches. Fix shape errors with `keepdims=True` or `[:, None]`.
- **Linear algebra** — `A @ B` matrix product (`*` is element-wise); `np.linalg.solve` not `inv(A) @ b`; `eigh` for symmetric; `svd` underlies PCA. Randomness: `np.random.default_rng(seed)` passed explicitly.
- **Series / DataFrame** — labelled 1-D / table of Series; operations **align on labels**. pandas 3: text columns are `str`, **Copy-on-Write** (every indexing result behaves as a copy; chained assignment silently does nothing).
- **`loc` vs `iloc`** — labels, **inclusive** end vs positions, exclusive end. `at` / `iat` for one cell.
- **Filters** — masks combined with `&`, `|`, `~`, each condition **parenthesised**; `and` / `or` → "truth value of a Series is ambiguous". Assign with one `.loc[mask, col] = value`.
- **Missing data** — find `isna().sum()`; `dropna(subset=…)`, `fillna`, group-median via `transform`, `ffill` / `interpolate` for time series.
- **groupby** — split–apply–combine: `agg` = one row per group; **`transform`** = same length as input (group stat broadcast back); `filter` = keep groups; `apply` = last resort.
- **merge** — SQL joins (`how=inner/left/right/outer/cross`); check with `validate=` and `indicator=True`. **`concat`** stacks.
- **Reshape** — `pivot_table` long → wide with aggregation; `melt` wide → long.
- **Dates** — parse early with an explicit format; `.dt` accessor; `resample` (calendar buckets, `"ME"`, `"h"` in pandas 3); `rolling` windows.
- **matplotlib** — OO style: `fig, ax = plt.subplots()`, draw on `ax`, label, `fig.savefig(path, dpi=…)`, `plt.close(fig)`. Line = trend, bar = compare, scatter = relationship, histogram = distribution.

Key facts & numbers

Performance tip	Why
column arithmetic, `np.where` / `np.select`, `.str` / `.dt`	compiled loops
`category` dtype, `float32` / `int32`	less memory, faster group-bys
`usecols=`, `dtype=`, `chunksize=`	read less
Parquet (needs pyarrow)	fast, columnar, keeps dtypes
built-in aggregations over `lambda` in `agg`	optimised code
collect then `concat` once	growing in a loop is quadratic
outgrown pandas	Polars, DuckDB, Dask, Spark

Format	Note
CSV	universal, slow, loses dtypes
Parquet	preferred for anything big
JSON (`lines=True`)	APIs, logs
SQL (`read_sql`)	via SQLAlchemy / sqlite3

Code to remember

```
df = pd.read_csv("orders.csv", parse_dates=["date"])
df["share"] = df["amount"] / df.groupby("customer")["amount"].transform("sum")
big = df.loc[(df["qty"] > 5) & (df["region"] == "EU"), ["customer", "qty"]]
m = orders.merge(customers, on="cust_id", how="left", indicator=True, validate="m:1")
orphans = m[m["_merge"] == "left_only"]
monthly = df.set_index("date")["amount"].resample("ME").sum()
```

In the interview

Q: Why is numpy fast? **A:** Contiguous typed memory, compiled loops, no per-element Python objects, SIMD, BLAS/LAPACK.

Q: `a * b` vs `a @ b`? **A:** Element-wise vs matrix product.

Q: Explain broadcasting. **A:** Shapes aligned from the right; equal or 1; 1 stretches.

Q: `loc` vs `iloc`? **A:** Labels with inclusive end vs positions with exclusive end.

Q: `transform` vs `agg`? **A:** One value per original row vs one per group.

Q: Orders with no matching customer? **A:** Left merge with `indicator=True`, filter `left_only`.

Q: How do you make pandas fast? **A:** Stay vectorised; avoid `apply(axis=1)` / `iterrows`; right dtypes; read less; Parquet.

Gotchas

- Expecting a slice to be a copy (it's a view) — or fancy indexing to be a view.
- `np.append` in a loop → O(n²); collect in a list.
- `==` on floats → `np.isclose` / `allclose`; mixing up `axis=0` / `axis=1`.
- `and` / `or` on Series; missing parentheses around conditions.
- Chained assignment `df[mask]["col"] = x` → ineffective in pandas 3.
- Merge row explosion from duplicate keys; key dtype mismatch (int vs str) → no matches.
- Mixing pyplot and OO calls; forgetting `plt.close(fig)` in loops; `plt.show()` in headless scripts; `twinx` for unrelated measures; rainbow colour maps for ordered data.

TL;DR

numpy: one dtype, views vs copies, broadcasting from the right, `@`, `solve`, seeded `default_rng`. pandas: `loc` inclusive/`iloc` exclusive, `&`/`|` masks, one `.loc` assignment, `groupby` + `transform`, validated merges, vectorise, Parquet. matplotlib: OO `fig, ax`, save and close.

19. Python for Coding Interviews

About twenty DSA patterns in Python, the built-in cost table, the Python idioms that make solutions short, and the traps that pass samples but fail hidden tests.

Must-know

- **State costs out loud** — list = dynamic array (O(1) end, O(n) front/search); dict/set O(1) avg; **slices and `+` copy** (O(k)); `sorted` O(n log n) time, O(n) space.
- **Size hints the target** — n ≤ 20 → exponential OK; n ≤ 10^3 → O(n^2); n ≤ 10^5–10^6 → O(n log n) or O(n).
- **Sliding window is O(n)** despite the inner `while` — each index enters and leaves once. It needs a monotonic rule; with **negative numbers** use prefix sums + hash map.
- **Monotonic stack/deque of indices** — O(n): each index pushed and popped once (next greater, span, window max).
- **Heap** — `heapq` is a **min-heap**; negate for max; top-k = size-k min-heap (root = weakest of the top k), O(n log k); add a tie-break counter.
- **Binary search** — `bisect_left([1,2,2,3], 2) == 1`, `bisect_right == 3`; half-open `lo < hi` template for "first true"; binary-search the **answer space** when feasibility is monotonic.
- **No TreeMap** built in — sorted list + `bisect` (O(n) insert), heap, or `sortedcontainers.SortedList` (third-party; preinstalled on some platforms).
- **Graphs** — `defaultdict(list)` adjacency; BFS marks visited **on enqueue**; Dijkstra skips stale heap entries and fails with negative edges; Kahn's topological sort also detects cycles.
- **Union-Find** — path compression + union by size → ~O(α(n)); components, undirected cycles, Kruskal (O(E log E)). Can't split groups or give paths.
- **DP** — write state + recurrence, memoise with `@cache` (top-down), convert to a table (bottom-up) if depth/memory matters, often compress to one row. Complexity = states × work per state. `cache` == `lru_cache(maxsize=None)`; `f.cache_info()`.
- **Backtracking** — choose → recurse → **undo**; store `path[:]`; sort + skip duplicates; sets for O(1) constraint checks (N-Queens diagonals: `r − c`, `r + c`).
- **Ints never overflow** — mask for 32-bit problems (`x & 0xFFFFFFFF`); `~x == -x - 1`.
- **Trie** — node = `dict` of children + end-of-word field; insert/search/prefix O(L); word search on a board = one trie + backtracking DFS that prunes non-prefixes.
- **Greedy** — take the locally best choice only if an exchange argument justifies it (coins {1, 3, 4} for 6 breaks naive greedy); intervals: sort by **end** to keep the most.
- **Intervals** — overlap iff `max(starts) <= min(ends)`; sort by start to merge; rooms needed = min-heap of end times or a +1/−1 sweep line.
- **Fast & slow** — meet ⇒ cycle; reset one to head, step both by 1 ⇒ cycle entry; also middle, happy number, find-duplicate (index → value). O(1) space.
- **Quickselect** — random-pivot partition, recurse into one side: k-th element in O(n) average (O(n^2) worst); Dutch flag sorts three values in one pass.

Key facts & numbers

Pattern table — signal → pattern → Python

Signal in the problem	Pattern	Python tool	Cost
sorted array, pair/triple sums	two pointers	`lo`, `hi` indices	O(n)
contiguous subarray/substring	sliding window	dict of last seen / counts	O(n)
range sums, "subarray sums to k"	prefix sums + hash map	`accumulate`, `defaultdict(int)`, `seen[0] = 1`	O(n)
seen-before, counts, groups, anagrams	hashing	`set`, `Counter`, `defaultdict(list)`, `tuple` keys	O(n)
brackets, next greater/smaller	(monotonic) stack	`list.append/pop`, `stack[-1]`	O(n)
FIFO, level order, window max	queue / monotonic deque	`deque.popleft/appendleft`	O(n)
k largest, merge k, running median	heap	`heapq` (negate for max)	O(n log k)
sorted search, min feasible value	binary search	`bisect_left/right`, `lo < hi`	O(log n)
linked list, tree	fast/slow, DFS/BFS	node classes, dummy head	O(n), O(h) space
unweighted / 0-1 / weighted shortest path	BFS / 0-1 BFS / Dijkstra	`deque` / `appendleft` / `heapq`	O(V+E) / O(V+E) / O((V+E) log V)
dependencies, ordering	topological sort (Kahn)	in-degree `Counter` + `deque`	O(V+E)
dynamic connectivity, MST	union-find	`parent` list, path compression	~O(α(n))
optimal value, overlapping subproblems	DP	`@cache` or table	states × work
all subsets/permutations/placements	backtracking	choose → recurse → undo	exponential
bits, parity, powers, number theory	bit tricks / math	`x & (x-1)`, `pow(b, e, m)`, `gcd`, `isqrt`, `comb`	O(1)–O(log)
streaming median, "best affordable" choice	two heaps	max-heap (negated) low half + min-heap high half	O(log n) per add
pick index with probability ∝ weight	prefix sums + binary search	`accumulate`, `bisect_right(prefix, r)`	O(n) build, O(log n) pick
store/rebuild a tree	pre-order + `#` for null	`",".join`, shared `iter(tokens)`	O(n)
prefixes, autocomplete, wildcard, words on a grid	trie	`dict` children + end-of-word field	O(L) per op
locally best choice is provably safe (jump game, gas station, candy)	greedy	one pass, often after `sorted(key=...)`	O(n) or O(n log n)
merge / insert / overlap ranges, meeting rooms	intervals	sort by start (merge) or end (keep most); `heapq` of ends; sweep line	O(n log n)
cycle entry, middle, palindrome list, duplicate in 1..n	fast & slow pointers	`slow`, `fast`, compare with `is`	O(n), O(1) space
k-th element, three values, sort a linked list	quickselect / Dutch flag / merge sort	random pivot partition; `low` / `mid` / `high`; fast/slow split	O(n) avg / O(n) / O(n log n)
spiral/rotate matrix, 32-bit reverse, collinear points, Josephus	math & geometry	shrinking boundaries; range check before `*10`; `gcd`-reduced slopes; `(J + k) % m`	O(r·c) / O(digits) / O(n²) / O(n)

Python-specific idioms

Need	Idiom
index + value / adjacent pairs	`enumerate(nums)` / `itertools.pairwise(nums)` (3.10)
running totals / running max	`accumulate(nums)` / `accumulate(nums, max)`
letter index	`ord(ch) - ord("a")`, back `chr(i + ord("a"))`
edit a string	`chars = list(s)` … `"".join(chars)`
count / top-k	`Counter(xs).most_common(k)` (heap, O(n log k))
anagram key	`tuple(sorted(w))` or `tuple(count26)`
multi-key sort	`sorted(ws, key=lambda w: (len(w), w))` ; negate for descending
queue with two stacks	move `inbox` → `outbox` when empty; amortised O(1)
infinity / big modulus	`math.inf` / `MOD = 10**9 + 7`
modular power / inverse	`pow(b, e, m)` / `pow(a, -1, m)` (3.8)
exact integer sqrt	`math.isqrt(n)` (not `math.sqrt`)
ceil division	`-(-a // b)`
bits	`x & -x` lowest bit, `x & (x-1)` clear it, `x.bit_count()` (3.10), `f"{x:08b}"`
power of two	`x > 0 and x & (x - 1) == 0` (`&` binds tighter than `==` in Python)

Algorithm choice for shortest paths: negative weights → Bellman–Ford O(V·E); all pairs, small V → Floyd–Warshall $O(V^3)$.

Code to remember

```python
def bfs(g, src):                        # unweighted shortest path
    dist, q = {src: 0}, deque([src])
    while q:
        u = q.popleft()
        for v in g[u]:
            if v not in dist:           # mark on enqueue
                dist[v] = dist[u] + 1; q.append(v)
    return dist
```

```python
def dijkstra(g, src):                   # g[u] = [(v, w), ...], w >= 0
    dist, pq = {src: 0}, [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist.get(u, math.inf): continue   # stale entry
        for v, w in g[u]:
            if d + w < dist.get(v, math.inf):
                dist[v] = d + w; heapq.heappush(pq, (d + w, v))
    return dist
```

```python
parent = list(range(n))
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]; x = parent[x]   # path halving
    return x
def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb: return False                          # already joined → cycle
    parent[ra] = rb; return True
```

In the interview

Q: How do you talk through a problem? **A:** Clarify (sizes, negatives, duplicates, empty) → examples → brute force + Big-O → name the pattern and why it's faster → code with idioms → test edge cases → time/space and follow-ups.

Q: Two-sum on unsorted data? **A:** Hash map of value → index, O(n)/O(n); sort + two pointers is O(n log n) and loses indices.

Q: Why a min-heap for k largest? **A:** The root is the weakest of the current top k — the one to evict.

Q: Running median? **A:** Two heaps (max-heap of low half via negation, min-heap of high half); O(log n) insert, O(1) median.

Q: Meeting rooms needed? **A:** Sort by start, keep a min-heap of end times; reuse the earliest room if it ended by this start; answer = heap size. Or sweep +1/−1 events with ends before starts at ties.

Q: When is greedy correct? **A:** When an exchange argument (or "greedy stays ahead") shows the local choice never hurts; otherwise use DP.

Q: k-th largest without sorting? **A:** Size-k heap O(n log k), or quickselect O(n) average with a random pivot (reorders the input).

Q: Where does a linked-list cycle start? **A:** After slow/fast meet, reset one to head; step both by 1; they meet at the start.

Q: Why no Dijkstra with negative edges? **A:** It finalises a node on pop; a later negative edge could shorten it.

Q: Top-down vs bottom-up DP? **A:** `@cache` is closest to the recurrence and computes only reachable states but hits the recursion limit; tables avoid it and allow row compression.

Q: Edit distance complexity? **A:** O(m·n) time; O(min(m, n)) space with rolling rows.

Q: Is using `Counter` / `heapq` / `bisect` allowed? **A:** Expected — but know each call's cost and be ready to hand-write it.

Gotchas

- `x in list` inside a loop → O(n²); `list.pop(0)` as a queue → `deque`.
- `helper(nums[1:])` in recursion copies O(n) per call → pass indices.
- Window length is `right − left + 1`; forgetting `seen[0] = 1` misses subarrays from index 0.
- `defaultdict` read with `d[k]` inserts → use `k in d`.
- `(priority, dict)` ties → `TypeError`; add a counter. Heap list isn't sorted. Negate on **both** push and pop.
- `bisect` on unsorted data silently wrong; `insort` is O(n).
- BFS marking visited on pop → duplicates/TLE; `visited` as a list; forgetting isolated nodes (loop `range(n)`).
- `@cache` on list arguments → unhashable; module-level `@cache` reused across test cases → stale answers (define inside, or `cache_clear()`).
- 0/1 knapsack iterated forwards becomes unbounded.
- `res.append(path)` instead of `path[:]`; forgetting to undo a choice.
- `if node.val:` when 0 is valid → `if node is not None`.
- Deep recursion (10^5-deep tree) → iterative stack; `sys.setrecursionlimit` last resort.
- `(a ** b) % m` → `pow(a, b, m)`; `math.sqrt` on huge ints → `isqrt`.
- Trie `search("ca")` true because the path exists — check the end-of-word field. Word search: restore the cell; clear `node.word` so a word is reported once.
- Intervals: `out[-1][1] = end` instead of `max(...)`; ask whether touching ends overlap.
- Dutch flag: don't advance `mid` after swapping with `high`. Quickselect/quicksort with a fixed pivot → O(n²) on sorted input.
- Slopes as floats or with un-normalised signs; spiral without the two `if` guards repeats a row/column.
- Version gaps on the judge: `pairwise` / `lcm` (3.9–3.10), `batched` (3.12), `heappush_max` (3.14) may be missing.

TL;DR

Map the signal to one of ~20 patterns, pick the Python structure with the right cost (set, deque, heapq, bisect, `@cache`), copy paths with `[:]`, mark BFS visited on enqueue, and narrate: clarify → brute force → optimise → code → test → complexity.

20. Pythonic Idioms, Gotchas and Interview Q&A

The idioms interviewers look for, the thirteen classic gotchas with their fixes, and a rapid-fire round. (Most of the chapter's ~45 Q&A are already folded into the chapters above.)

Must-know

- **Pythonic** = use the protocols: iterate directly (`enumerate`, `zip`), build with comprehensions, test truthiness, `with` for every resource, f-strings, `pathlib`, and **EAFP with specific exceptions** (also avoids check-then-use races).
- **Generator vs list** — generator when consumed once (`sum`, `any`), list when you need to index, reuse or `len()` it.
- **How to answer** — one-sentence definition → mechanism → use case or trade-off; show the edges ("O(1) average, O(n) worst", "except the free-threaded build"); a tiny example; say "I'd check the docs" rather than invent.

Key facts & numbers

Instead of…	Write…
`if key in d: v = d[key] else: v = default`	`v = d.get(key, default)`
`tmp = a; a = b; b = tmp`	`a, b = b, a`
`for i in range(len(xs))`	`for i, x in enumerate(xs)`
parallel indexing	`for x, y in zip(a, b, strict=True)`
loop + `append`	`[f(x) for x in xs if cond(x)]`
`if len(xs) == 0` / `if flag == True`	`if not xs` / `if flag`
`"Hi " + name + str(age)`	`f"Hi {name}, {age}"`
`f = open(p) … f.close()`	`with open(p, encoding="utf-8") as f:`
`os.path.join(d, "x.txt")`	`Path(d) / "x.txt"`
`s += piece` in a loop	`"".join(pieces)`
`x == None`	`x is None`
flag variable + loop	`for … else` or `any(...)`

Rapid-fire	One-line answer
`range` in Python 3?	lazy immutable sequence; O(1) `len` and `in` for ints
`del` vs `remove` vs `pop`?	by index/slice/name; by value (first match); by index, returns item
What is `None`?	the single `NoneType` instance; test with `is None`
`copy.copy` on a tuple of ints?	returns the same tuple
`*` / `/` alone in a signature?	keyword-only after / positional-only before
`enumerate` returns?	lazy iterator of `(index, item)`
`cache` vs `lru_cache`?	unbounded vs size-limited memoisation
`zip([1, 2, 3], "ab")`?	`[(1, 'a'), (2, 'b')]` — stops at shortest
In-place methods returning `None`?	`list.sort`, `list.append`, `random.shuffle`, `dict.update`
`"" or "default"`, `0 and 1`?	`'default'`, `0` — operands, not bools

Code to remember

```python
def add_item(item, bucket=None):        # gotcha 1 fix
    if bucket is None:
        bucket = []
    bucket.append(item); return bucket

fns = [lambda i=i: i for i in range(3)] # gotcha 2 fix → [0, 1, 2]
nums = [n for n in nums if n != 2]      # gotcha 5 fix
```

In the interview

Q: Name the classic Python gotchas. **A:** Mutable defaults, late-binding closures, `is` vs `==`, shallow copies, mutating while iterating, float precision, bare `except` — each with its fix below.

Q: Why are defaults evaluated once? **A:** They're part of the function object, created when `def` runs; re-evaluating per call would be slower and change semantics.

Q: Capture the current loop value in a closure? **A:** `lambda i=i: i`, `functools.partial`, or a factory function.

Q: EAFP or LBYL to open a file that may not exist? **A:** EAFP — `try: open(...) except FileNotFoundError:`; the file could vanish between check and open.

Q: `a = [[0] * 2] * 2; a[0][0] = 1` → ? **A:** `[[1, 0], [1, 0]]`.

Q: What does bare `except:` catch that `except Exception:` doesn't? **A:** `KeyboardInterrupt`, `SystemExit`, `GeneratorExit`.

Gotchas

1. **Mutable default arguments** — `def f(item, bucket=[])` shares one list (stored in `__defaults__`) → default `None`, create inside.
2. **Late-binding closures** — `[lambda: i for i in range(3)]` all return 2 (variable looked up at call time) → `lambda i=i: i` or `partial`.
3. **`is` vs `==`** — identity vs value → `==` for values; `is` only for `None` and sentinels.
4. **Integer caching / interning** — `256 is 256` True, `257 is 257` may be False (−5…256 are pre-created; literals in one code block may be shared) → never rely on it.
5. **Mutating while iterating** — removing from a list skips the next item; dicts/sets raise `RuntimeError: … changed size during iteration` → comprehension, iterate a copy (`nums[:]`, `list(d)`).
6. **Shallow copy** — `copy()`, `list(x)`, `x[:]`, `copy.copy` share inner objects; `alias = x` copies nothing; `[[0]*n]*m` shares rows → `deepcopy`, `[row[:] for row in grid]`, comprehension-built grids.
7. **Shared mutable values** — `dict.fromkeys(keys, [])` shares one list; any `dd[key]` on a `defaultdict` **inserts** → dict comprehension; test with `key in dd` or `dd.get`.

8. **Class vs instance attributes** — `tricks = []` in the class body is shared by all instances; *assigning* through an instance shadows instead → mutable state in `__init__`.
9. **Circular imports** — the second module sees a partially initialised first one: `ImportError: cannot import name …` → move shared code to a third module; `import mod` and use `mod.name` at call time; import inside the function; `if TYPE_CHECKING:` for hint-only imports.
10. **Floating point** — `0.1 + 0.2 == 0.3` False; `round(2.675, 2) == 2.67` ; `round(0.5), round(1.5) == 0, 2` (half-to-even); `nan != nan` ; `Decimal(0.1)` is inexact → `math.isclose` (with `abs_tol` near zero), `Decimal("0.1")`, `Fraction`, integer cents.
11. **Bare `except:`** — swallows `KeyboardInterrupt` / `SystemExit` and real bugs → catch the narrowest exception; log and re-raise the rest.
12. **Shadowing built-ins** — `list = [...]` then `list("abc")` → `'list' object is not callable` (LEGB) → names like `items`, `total`.
13. **Surprising return values** — in-place methods return `None` (`xs = xs.sort()` loses the list); `and` / `or` return operands.

- Also on the list: `//` floors toward −∞ (`-7 // 2 == -4`); recursion limit ~1000 with no TCO; `if value:` treats `0` / `""` as missing; `"\b"` without `r` is a backspace; `zip` silently truncates; generators are one-shot.

TL;DR

Defaults once, closures late, `is` for identity, copies shallow, don't mutate while iterating, floats inexact, `fromkeys` / `defaultdict` share and insert, class attributes are shared, break import cycles, catch specific exceptions, don't shadow built-ins, in-place methods return `None`.

Last-minute checklist

- Names are labels on objects; `=` never copies; `is` only for `None`.
- Know which types are mutable (list, dict, set, bytearray) and which are hashable.
- `0.1 + 0.2 != 0.3` → `isclose` ; money → `Decimal("…")` ; `round` is half-to-even; `//` floors toward −∞.
- Truthiness: `None`, `False`, zeros, empties are falsy; use `is not None` when 0 is valid.
- List = dynamic array (O(1) end, O(n) front and `in`); dict/set = hash table O(1) avg; deque O(1) both ends.
- Timsort: stable, O(n log n), `key` called once per item; `sort()` returns `None`.
- Strings are immutable → `"".join(parts)` ; f-strings; `open(..., encoding="utf-8")`.
- Defaults evaluated once → `None` ; closures bind late → `i=i` ; LEGB + assignment makes local.
- Decorator = `f = d(f)` + `@wraps` ; `with` = enter/body/exit, truthy exit suppresses.
- Iterables restart, iterators/generators are one-shot; `groupby` needs sorted input.
- `__repr__` first; `__eq__` and `__hash__` agree; return `NotImplemented` ; `super()` follows the C3 MRO.
- Dataclass mutable defaults via `field(default_factory=list)` ; `frozen=True` is shallow.
- Specific `except`s, bare `raise` to re-throw, `raise X from e` ; `assert` is stripped by `-O`.
- Imports run once (`sys.modules`); first match on `sys.path` wins; one venv per project, `python -m pip`.
- Type hints are never enforced; narrow `X | None` ; validate untrusted data with pydantic.
- GIL: I/O → threads/asyncio, CPU → processes/numpy; `x += 1` is not atomic; `TaskGroup` over `gather`.
- Refcount + cycle GC; profile before optimising; set lookups beat list scans.
- numpy slices are views; pandas `loc` inclusive, `iloc` exclusive; masks with `&` / `|` in parentheses.
- Coding round: clarify → brute force + Big-O → name the pattern → code → edge cases → complexity.
- Patterns: two pointers, window, prefix sums, hashing, stack/deque, heap, binary search, BFS/Dijkstra/topo, union-find, DP, backtracking, bits.
- `deque` for BFS (mark visited on enqueue), `heapq` negated for max-heap with tie-breakers, `path[:]` in backtracking, `pow(b, e, m)`, `isqrt`.